# A High-Performance Vectorized Framework for Multiobjective Evolutionary Optimization in Python

Nicolás R. Uribe, Alberto Herrán, Antonio J. Nebro, Javier Del Ser, and J. Manuel Colmenar

***Abstract*—Python is a widely adopted platform for empirical studies in multi-objective optimization, yet its interpreted nature makes metaheuristics implemented in pure Python inherently slower than those of frameworks written in compiled languages. Most Python libraries further aggravate this limitation by managing populations as collections of individual solution objects, which prevents effective vectorization and keeps hot-path computations in the interpreter. We present VAMOS (Vectorized Architecture for Multiobjective Optimization Studies), a Python framework that stores population state in dense numerical arrays and dispatches hot-path computations to pluggable backends (NumPy, Numba, MooCore/C), keeping algorithmic logic independent of the numerical substrate. VAMOS implements nine multi-objective evolutionary algorithms spanning dominance-based, decomposition-based, and indicator-based paradigms, with richer component-level configurability than typical frameworks, including interchangeable variation operators, configurable steady-state modes, and optional bounded or unbounded external archives. A two-line entry point, automatic algorithm selection, and a browser-based graphical interface lower the entry barrier for domain experts with limited Python experience, while explicit component-level configuration supports metaheuristics practitioners who require fine-grained control. On the ZDT, DTLZ, and WFG benchmark suites with two and three objectives, VAMOS with its Numba backend reduces wall-clock runtime by 4–12× relative to object-centric libraries and by 1.1–1.2× relative to the closest array-aware competitor, with the advantage widening as population size grows. Paired Wilcoxon signed-rank tests with Holm correction confirm statistically equivalent solution quality across frameworks. VAMOS is an open-source project available at https://github.com/vamos-framework/vamos.**

***Index Terms*—Benchmarking, Evolutionary algorithms, Multiobjective optimization, Software framework, Vectorization**

## I. Introduction

MULTI-OBJECTIVE optimization problems (MOPs) arise whenever trade-offs exist between conflicting objectives, requiring the computation of an accurate approximation, in terms of convergence and diversity, of the Pareto-optimal set rather than a single optimum [1]. Such problems are pervasive in engineering design, machine learning, scheduling, and resource allocation, among other domains. In this field, multi-objective evolutionary algorithms (MOEAs) remain a dominant approach due to their robustness, population-based anytime behavior, and ability to return a set of non-dominated solutions in a single run [2].

The widespread adoption of MOEAs has been fueled in large part by the availability of software frameworks that lower the implementation barrier and facilitate reproducible experimentation. Early efforts such as PISA [3] established the concept of modular, platform-independent interfaces for multi-objective search. The jMetal framework [4] later consolidated a comprehensive Java-based toolkit that became a reference for algorithm development and research studies. More recently, PlatEMO [5] has provided a MATLAB platform with a large catalog of algorithms and test problems. In parallel, the Python ecosystem has seen the emergence of dedicated libraries, such as pymoo [6], jMetalPy [7], Platypus [8], and DEAP [9], that leverage Python's accessibility and rich scientific stack.

As Python has become a widely adopted platform for MOEA research and experimentation, three practical limitations hinder its broader adoption. First, Python's interpreted nature makes the inner loops of pure-Python implementations inherently slower than those of frameworks written in compiled languages such as C++ or Java, and many Python MOEA libraries further aggravate this overhead by managing populations as collections of per-solution objects, preventing effective vectorization and keeping hot-path computations, namely, selection, variation, and survival, inside the interpreter [7]–[9].

Second, existing MOEA frameworks often provide limited configurability at the component level. If we focus on NSGA-II [10], for instance, most tools expose only commonly used parameters, such as the population size, the number of generations, or the crossover and mutation probabilities and distribution indexes of the SBX and polynomial mutation operators. However, features such as setting the offspring population size (e.g., to configure a steady-state variant if that size is 1) or attaching external archives, which have been shown to be useful to allow NSGA-II to handle problems with more than two objectives [11], are not included in most tools, with the exception of jMetal [4].

Third, adoption is hindered by usability gaps affecting two complementary user profiles. On the one hand, domain experts who face multi-objective trade-offs in fields such as biology, physics, or engineering may have limited experience with Python or optimization frameworks; in this case, the need to import and wire together multiple modules (algorithm, problem, operators, termination), understand framework-specific abstractions, and write 10–15 lines of boilerplate code for a standard run becomes a practical barrier. On the other hand, users who are expert in metaheuristics benefit from frame-

N. R. Uribe, A. Herrán, and J. M. Colmenar are with the Department of Computer Sciences, Universidad Rey Juan Carlos, Móstoles, 28933 Madrid, Spain (e-mail: nicolas.rodriguez@urjc.es; alberto.herran@urjc.es; josemanuel.colmenar@urjc.es).

A. J. Nebro is with the Department of Lenguajes y Ciencias de la Computación, ITIS Software, University of Málaga, 29071 Málaga, Spain (e-mail: ajnebro@uma.es).

J. Del Ser is with TECNALIA, Basque Research & Technology Alliance (BRTA), Derio, 48160, Spain, and also with the University of the Basque Country (UPV/EHU), Leioa, 48940, Spain (e-mail: javier.delser@tecnalia.com).

Corresponding author: Alberto Herrán (alberto.herran@urjc.es).

works that expose explicit control over algorithm components and hyperparameters so that they can exploit the full potential of MOEAs without being constrained by rigid presets.

We introduce VAMOS, a Python-based package designed to address these limitations, i.e., runtime overhead, configurability, and accessibility, with a single guiding principle: *keep algorithm state in dense arrays and push hot loops into vectorized kernels*. VAMOS stores population state in dense numerical arrays and dispatches hot-path computations to pluggable compute backends (NumPy, Numba, MooCore/C), keeping algorithmic logic independent of the numerical substrate.

To support both novice and expert workflows, VAMOS provides a two-line entry point (`optimize("zdt1", algorithm="nsgaii")`) for complete optimization runs, a `make_problem` factory that turns any Python function into a problem without subclassing, automatic algorithm selection from problem traits, and a browser-based graphical interface (*VAMOS Studio*) with domain-specific templates that allows users to define and solve problems without writing any code. The contributions of this work are:

1) **High-performance vectorized core**: a unified, array-based internal representation with pluggable compute kernels (NumPy/Numba/MooCore) that reduces inner-loop overhead by $4$–$12\times$ relative to object-centric libraries.
2) **Rich component-level configurability**: nine MOEAs implemented on top of shared components, with interchangeable variation operators, configurable offspring batch size (enabling steady-state variants), optional bounded or unbounded external archives, multiple constraint-handling modes.
3) **Accessible API and tooling for novice-to-expert workflows**: a two-line entry point for complete optimization runs, a `make_problem` factory that turns any Python callable into a problem without subclassing, automatic algorithm selection from problem traits, a CLI wizard for guided setup, and a Web-based interactive dashboard (*VAMOS Studio*) with domain-specific templates for both novice and expert users.

The rest of the paper is organized as follows. Section II reviews existing Python MOEA frameworks and benchmarking practices. Section III presents the VAMOS framework, including its design goals, architecture, compute kernels, and algorithm suite. Section IV describes the experimental setup, reports backend-level and cross-framework runtime comparisons including a scalability study of the vectorized architecture, and provides solution-quality summaries with statistical analyses. Finally, Section V draws conclusions and outlines directions for future work.

## II. Related Work

We review the most relevant Python MOEA frameworks below.

### A. Python MOEA frameworks

Several mature Python libraries exist for MOEAs. We describe next the most relevant ones, including pymoo, jMetalPy, DEAP, Platypus, and Pygmo:

**pymoo** [6] is arguably the most comprehensive Python framework currently available, offering a wide range of multi-objective algorithms (NSGA-II, NSGA-III, MOEA/D, among others), alongside utilities for performance indicator computation, visualization, and constraint handling. Its object-oriented architecture encourages algorithm customization through subclassing, and its documentation and community support are well-established. However, pymoo's design tightly couples algorithmic logic with its internal data structures, making it difficult to swap computation backends or to systematically push hot-path computations into compiled kernels. Component-level configurability, such as interchangeable variation operators or configurable archiving strategies, requires non-trivial subclassing efforts.

**jMetalPy** [7] is the Python port of the widely used jMetal Java framework [4] and follows a similar object-oriented, component-based design philosophy. It supports both standard and preference-based multi-objective algorithms and includes observer and observable patterns for runtime monitoring. Despite these strengths, jMetalPy's architecture prioritizes design clarity and Java-style extensibility over computational performance, resulting in slower runtimes compared to array-oriented frameworks. Its population representation relies on lists of solution objects rather than dense numerical arrays, which limits the extent to which hot-path computations can be vectorized or accelerated by alternative computation backends.

**DEAP** (Distributed Evolutionary Algorithms in Python) [9] takes a highly generic approach, providing a toolkit of evolutionary operators that can be assembled into custom algorithms through a combination of functional programming patterns and a declarative toolbox mechanism. This makes DEAP extremely flexible for prototyping novel algorithms. However, this generality comes at the cost of abstraction: multi-objective optimization is supported through NSGA-II and SPEA2, but building more sophisticated pipelines requires substantial user effort. DEAP does not provide integrated support for performance indicators, external archives, or decomposition-based paradigms, and its per-individual object model shares the same vectorization limitations as jMetalPy.

**Platypus** [8] is a lightweight framework specifically designed for multi-objective optimization, implementing several classical algorithms including NSGA-II, NSGA-III, MOEA/D, and epsilon-MOEA. Its API is clean and beginner-friendly, and it supports parallelism via Python's multiprocessing module. Platypus, however, has seen limited maintenance in recent years and lacks support for more recent indicator-based algorithms and advanced archiving strategies. Like jMetalPy and DEAP, it relies on object-based population representations that preclude backend-level vectorization.

**pygmo** [12] is a Python library wrapping the C++ optimization framework pagmo2, designed with performance and parallelism as primary concerns. By exposing its core algorithms through compiled C++ extensions, pygmo achieves runtimes

that are substantially faster than pure-Python frameworks, and its island model provides a principled abstraction for distributed and parallel optimization. pygmo supports several multi-objective algorithms, including NSGA-II and MOEA/D, and allows users to define custom problems and algorithms through a duck-typing interface. However, pygmo's architecture is primarily oriented toward single-objective and continuous optimization, and its multi-objective capabilities are comparatively limited in scope. Furthermore, pygmo does not expose component-level configurability in variation operators and selection mechanisms, and archiving strategies are bundled within each algorithm and cannot be recombined independently, which limits its utility for practitioners interested in algorithm design rather than algorithm application.

Taken together, these frameworks occupy a spectrum between usability and performance. To the best of our knowledge, they do not explicitly separate algorithmic logic from the numerical substrate in a way that enables transparent substitution of computation backends, nor do they combine a low-barrier entry point for domain experts with fine-grained component configurability for practitioners, which motivates the focus of VAMOS.

In contrast to these Python libraries, EvoX [13] targets automated, distributed, and heterogeneous execution of general evolutionary computation workloads. It provides a functional programming model for declaring the logical flow of evolutionary algorithms and a hierarchical state-management strategy that maps the resulting workflow onto heterogeneous resources, enabling GPU-accelerated and multi-node execution. In contrast, VAMOS targets a complementary, algorithm-level gap: a vectorized, backend-agnostic architecture optimized for multi-objective optimization studies, with emphasis on efficient inner-loop execution in Python and reproducible cross-framework benchmarking.

Beyond general-purpose MOEA libraries and infrastructure-oriented frameworks like EvoX, recent research has introduced specialized frameworks and workflows. Examples include a coevolutionary approach for constrained multiobjective optimization using helper problems and information sharing [14]; an evolutionary multitask framework for multimodal optimization with explicit/implicit bi-knowledge transfer between species [15]; Light-EvoOPT, a four-stage pipeline for ultralarge-scale mixed-integer linear programs combining problem division, model-based initialization, reduction, and evolutionary improvement [16]; and LLaMoCo, which instruction-tunes large language models to generate optimization code [17]. These contributions focus on specific problem settings or automation, whereas VAMOS targets a vectorized, backend-agnostic architecture for multi-objective optimization studies and reproducible cross-framework benchmarking.

Table I summarizes the main differences between Python multi-objective optimization frameworks relevant to this work, including algorithmic capabilities, architectural features, configurability, accessibility (minimum lines of code for a standard run), and tooling. VAMOS is the only framework that combines a dense array population model, pluggable computation backends, rich component-level configurability, a minimal two-line entry point (Listing 1), and a graphical user interface.

Fig. 1 provides an intuitive view of how these frameworks represent and manipulate a population. In VAMOS, the population state is stored in dense arrays ($X \in \mathbb{R}^{N\times n}$ and $F \in \mathbb{R}^{N\times m}$), and selection/variation/survival primarily operate by computing index arrays and applying array kernels. In contrast, pymoo and jMetalPy typically manage the population as a collection of solution objects; array computations may still be used internally, but more of the algorithmic control flow involves object/container manipulation.

## III. VAMOS Framework

This section presents the VAMOS framework. We first state the design goals that guided its development (Section III-A), then describe the architecture and data model (Section III-B), compute kernels, algorithm suite, execution model, variation pipeline, archives and metrics, and the benchmark runner.

### A. Design goals

The design of VAMOS is motivated by three practical gaps identified in existing Python MOEA libraries, corresponding to the three contributions of this work: performance, configurability, and accessibility.

*a) Performance: array-native execution with pluggable backends.:* The population state remains in dense arrays throughout the optimization loop, and hot-path computations are dispatched to vectorized or compiled kernels. Switching the compute backend is a single-parameter change (`engine="numba"`, `"numpy"`, etc.) that is transparent to algorithm code. No other Python MOEA framework allows swapping the entire computational substrate—from array operations to compiled kernels—without modifying the algorithm implementation. Similarly, evaluation strategies (serial, multiprocessing, or Dask-based distributed evaluation) are selected via configuration rather than code changes, and multi-seed studies are first-class: passing a list of seeds (`seed=[0,1,2]`) triggers automatic multi-run execution and returns a list of results. Section IV demonstrates that this design yields substantial runtime reductions across multiple MOEA variants and benchmark families without degrading solution quality.

*b) Configurability: fine-grained control over algorithmic components.:* Most frameworks provide a small set of high-level parameters (population size, crossover/mutation probabilities), but do not expose fine-grained control over offspring batch size, archive attachment, or operator adaptation. VAMOS exposes all of these as first-class configuration options through a builder API (Listing 2 in Appendix F): users can set the offspring population size (e.g., to configure a steady-state variant), attach bounded or unbounded external archives, and select among multiple constraint-handling modes (feasibility rules, penalty functions, $\varepsilon$-relaxation). Algorithm performance is also sensitive to hyperparameters; VAMOS therefore includes a tuning module (`vamos.engine.tuning`) exposed both programmatically and via the CLI (`vamos tune`). It supports random search and an irace-inspired racing procedure [18] with multi-fidelity evaluation (successive halving), warm-starting, and parallel execution. The same interface can

TABLE I
COMPARISON OF PYTHON MULTI-OBJECTIVE OPTIMIZATION FRAMEWORKS. ✓ INDICATES FULL SUPPORT, ❍ INDICATES PARTIAL OR LIMITED SUPPORT, AND ✗ INDICATES ABSENCE OF THE FEATURE.

| Feature | VAMOS | pymoo | jMetalPy | DEAP | Platypus | pygmo |
|---|---|---|---|---|---|---|
| Dominance-based algorithms | ✓ | ✓ | ✓ | ❍ | ✓ | ❍ |
| Decomposition-based algorithms | ✓ | ✓ | ❍ | ✗ | ✓ | ❍ |
| Indicator-based algorithms | ✓ | ❍ | ❍ | ✗ | ✗ | ✗ |
| Pluggable computation backends | ✓ | ✗ | ✗ | ✗ | ✗ | ✗ |
| Dense array population model | ✓ | ❍ | ✗ | ✗ | ✗ | ✗ |
| Interchangeable variation operators | ✓ | ❍ | ❍ | ✓ | ❍ | ✗ |
| Configurable steady-state mode | ✓ | ✗ | ❍ | ❍ | ✗ | ✗ |
| External archive support | ✓ | ✗ | ✗ | ❍ | ❍ | ✗ |
| Performance indicators | ✓ | ✓ | ❍ | ✗ | ❍ | ✗ |
| Parallelism | ❍ | ❍ | ❍ | ✓ | ❍ | ✓ |
| Graphical user interface | ✓ | ✗ | ✗ | ✗ | ✗ | ✗ |
| Automatic algorithm selection | ✓ | ✗ | ✗ | ✗ | ✗ | ✗ |
| Min. lines of code† | 2 | ∼10 | ∼15 | ∼25 | ∼8 | – |
| Active maintenance | ✓ | ✓ | ❍ | ❍ | ✗ | ✓ |

†Lines of code for a minimal NSGA-II run on a built-in benchmark.

also delegate to model-based backends such as Optuna [19], SMAC3 [20], and BOHB [21], which are provided as optional dependencies.

*c) Accessibility: lowering the entry barrier for domain experts.:* Most frameworks require importing separate modules for the algorithm, problem, operators, and termination, then wiring them together (typically 10–15 lines for a standard Nondominated Sorting Genetic Algorithm II (NSGA-II) run on a benchmark problem). This level of boilerplate assumes familiarity with MOEA terminology and framework internals, which is a barrier for domain experts in fields such as biology, physics, or engineering who need multi-objective optimization but lack a background in metaheuristics. VAMOS addresses this gap at multiple levels. First, a single entry point (`optimize`) handles defaults, operator configuration, and termination in a single call (Listing 1 in Appendix F): two lines of code are sufficient to run a complete optimization with sensible defaults. For comparison, the equivalent run in pymoo requires ∼10 lines; jMetalPy requires ∼15 lines. Second, a `make_problem` factory turns any Python callable into a problem object without subclassing (Listing 3), and an interactive CLI wizard (`vamos create-problem`) scaffolds a ready-to-run script from a guided questionnaire. Third, VAMOS provides automatic algorithm selection (`algorithm="auto"`) that maps problem traits (number of objectives, encoding type) to a suitable default algorithm. When finer control is needed, the builder API ensures that the framework scales from novice to expert use without a different API.

## B. Architecture and data model

VAMOS is organized into four layers:

1) **Foundation**: problem definitions (ZDT, DTLZ, WFG, ZCAT, CEC2009, LZ09, LSMOP, constrained families such as C-DTLZ and MW, and real-world suites), kernels, metrics, and archives;
2) **Engine**: algorithm implementations and shared components;
3) **Experiment**: CLI, benchmarking utilities, visualization, and reporting;
4) **UX**: interactive dashboard (*VAMOS Studio*) with a visual problem builder and results explorer.

Fig. 2 illustrates the core layered organization: the Foundation layer provides problem definitions, compute kernels, metrics, and archives; the Engine layer implements nine MOEAs on top of shared components; the Experiment layer exposes CLI tools, benchmarking utilities, and reporting; and the UX layer provides VAMOS Studio, a Streamlit-based interactive environment for visual problem definition, live Pareto-front preview, and post-run analysis.

The user-facing API and configuration model are described in Section III-A. The core design choice is to represent a population as a pair of dense arrays: decision variables $X \in \mathbb{R}^{N \times n}$ and objective values $F \in \mathbb{R}^{N \times m}$ for population size $N$, $n$ decision variables, and $m$ objectives. Variation and survival operate on these arrays (or views thereof), enabling vectorized operations (NumPy) and compilation of hot loops (Numba) without per-individual Python objects.

## C. Compute kernels

Algorithmic logic (e.g., *NSGA-II selects survivors by nondominated sorting and crowding distance*) is separated from numerical kernels that implement the corresponding computations. VAMOS provides multiple backends:

- **NumPy**: baseline backend using vectorized array operations [22].
- **Numba**: just-in-time (JIT) compilation of hot operators and utilities, reducing Python overhead [23].
- **MooCore**: optional C-backed kernels for Pareto ranking and hypervolume-based utilities [24].

Backends are selected through configuration (e.g., `engine="numba"`) and are transparent to algorithm code.

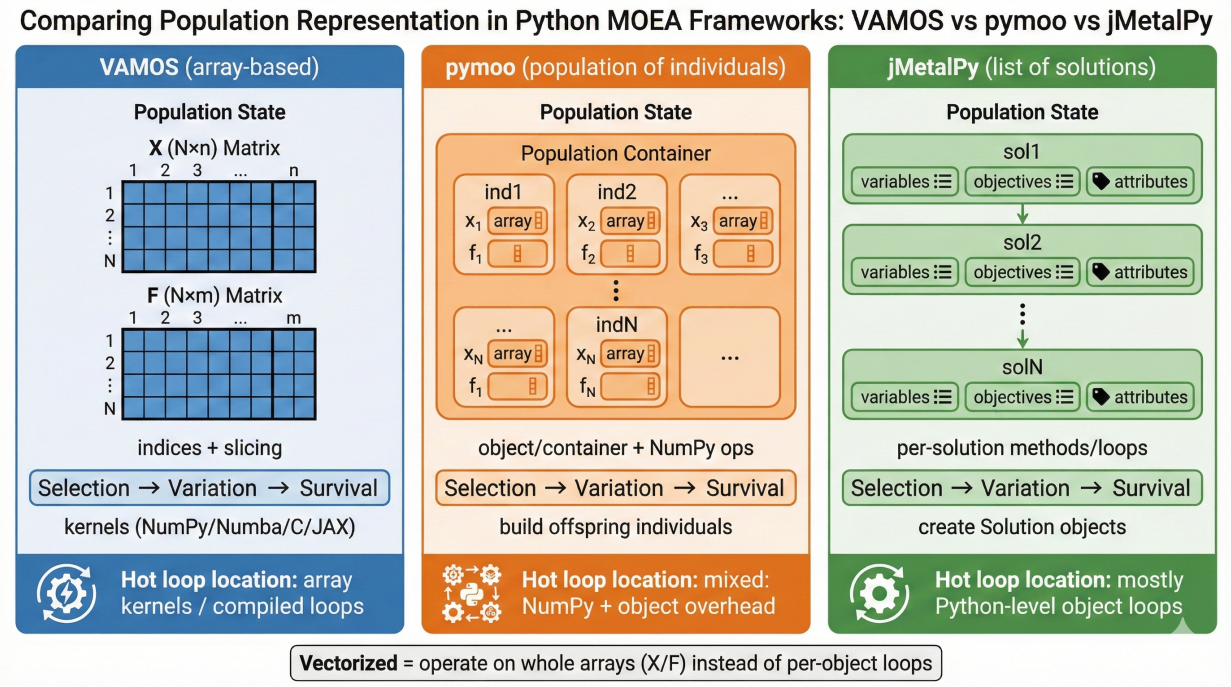


Fig. 1. Comparing population representation across Python MOEA frameworks. VAMOS (left) stores the full population in dense arrays ($X$, $F$) and applies operators via array-oriented kernels (NumPy/Numba/MooCore). pymoo (center) uses a population container of individual objects backed by NumPy arrays, mixing array computation with object overhead. jMetalPy (right) manages a list of solution objects with per-solution method calls. All three implement the same selection–variation–survival loop, but differ in internal data model and hot-loop granularity.

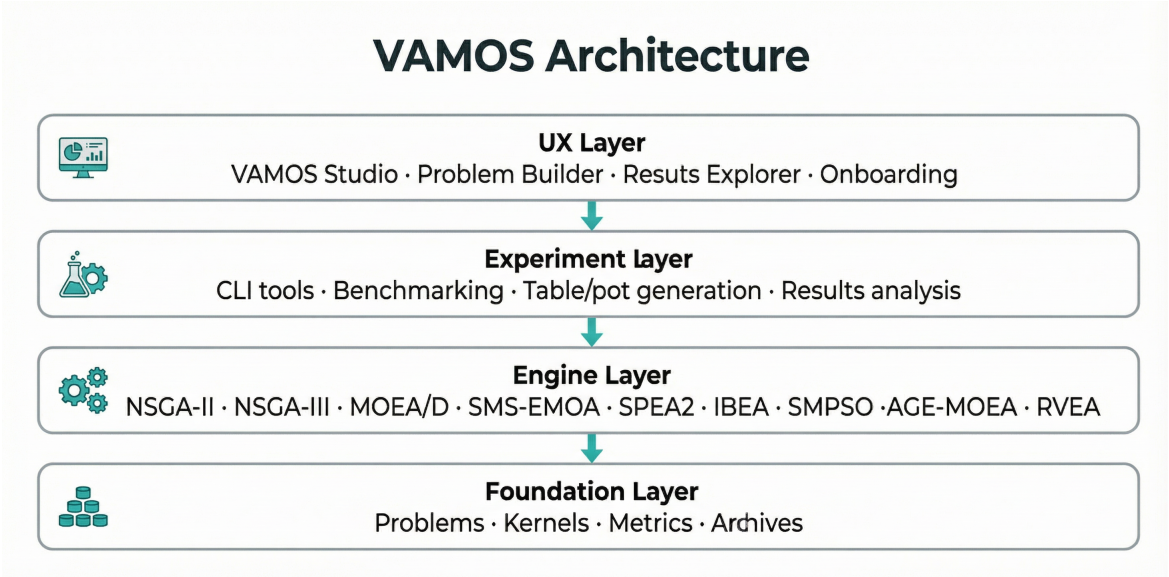


Fig. 2. VAMOS four-layer architecture. The Foundation layer provides problem definitions, compute kernels (NumPy, Numba, MooCore/C), metrics, and archives. The Engine layer implements nine MOEAs using shared selection, variation, and survival components. The Experiment layer exposes CLI tools for problem creation, hyperparameter tuning, benchmarking runners, table/plot generation, and results analysis. The UX layer provides VAMOS Studio, an interactive dashboard for visual problem building, onboarding, and results exploration.

In addition to accelerating arithmetic kernels, VAMOS reduces overhead by minimizing Python-side allocations: variation operators can reuse pre-allocated workspaces (e.g., scratch buffers for simulated binary crossover/polynomial mutation) and algorithms maintain state as contiguous arrays. This design is particularly beneficial in inner-loop routines such as tournament selection, non-dominated sorting, crowding-distance computation, and archive updates.

### D. Algorithm suite

VAMOS implements nine multi-objective algorithms covering dominance-based, decomposition-based, indicator-based, and reference-vector paradigms. Table II summarizes the suite.

TABLE II
MULTI-OBJECTIVE ALGORITHMS IMPLEMENTED IN VAMOS

| Algorithm | Category | Ref. |
|---|---|---|
| Nondominated Sorting Genetic Alg. II (NSGA-II) | Dominance | [10] |
| Nondominated Sorting Genetic Alg. III (NSGA-III) | Dominance | [25] |
| MOEA Based on Decomposition (MOEA/D) | Decomposition | [26] |
| S-Metric Selection EMOA (SMS-EMOA) | Indicator | [27] |
| Strength Pareto EA 2 (SPEA2) | Dominance | [28] |
| Indicator-Based EA (IBEA) | Indicator | [29] |
| Speed-Constrained PSO (SMPSO) | Swarm | [30] |
| Adaptive Geometry Estimation MOEA (AGE-MOEA) | Dominance | [31] |
| Reference Vector Guided EA (RVEA) | Decomposition | [32] |

All nine algorithms are built on top of the shared selection, variation, and survival components described in Sections III-B and below, ensuring consistent operator semantics and enabling component-level substitution (e.g., replacing SBX crossover and polynomial mutation with any of the others available variation operators and using bounded and unbounded external archives) without algorithm-specific code changes. The suite spans the three major MOEA paradigms: dominance ranking (NSGA-II, NSGA-III, SPEA2, AGE-MOEA), decomposition (MOEA/D, RVEA), and indicator contribution (SMS-EMOA, IBEA), plus a particle swarm optimization algorithm (SMPSO), so that practitioners can compare paradigms under identical infrastructure and configuration conventions. Table XX in Appendix E summarizes the configurable parameter space for each algorithm, ranging from 8 parameters (SMPSO) to 37 (NSGA-II).

### E. Execution model and evaluation backends

Algorithms in VAMOS expose a uniform `run(problem, termination, seed, eval_backend, live_viz)` interface, which the high-level `optimize()` entry point (Section III-A) delegates to after resolving defaults and configuring operators. Several algorithms also support an *ask–tell* loop for streaming or interactive use. Objective evaluations are dispatched through a small evaluation-backend protocol, enabling serial evaluation for controlled benchmarking and parallel evaluation when objective functions are expensive. Separating evaluation from algorithmic state keeps the algorithm core deterministic under a fixed random seed and reduces the risk of framework-specific parallelism becoming a confounder in empirical comparisons.

### F. Variation pipeline and encodings

VAMOS supports real-valued, binary, integer, permutation, and mixed-variable encodings through a variation pipeline that composes selection, crossover, mutation, and optional repair operators. For continuous domains, the default configuration uses simulated binary crossover (SBX) and polynomial mutation (PM), with mutation probability typically set to $1/n$ where $n$ is the number of decision variables. Binary and integer encodings use standard crossover (one-point, two-point, or uniform) and bit-flip or bounded mutation, while permutation encodings provide order and cycle crossover with insertion mutation. Mixed-variable problems combine encoding-specific operators transparently. Encodings are normalized internally to ensure consistent operator resolution and avoid silent configuration mismatches. The experimental evaluation in Section IV focuses on continuous encodings; benchmarking of discrete and mixed-variable operators is left to future work.

### G. Archives, metrics, and analysis tooling

The framework includes optional external archives—both bounded (with crowding-distance or hypervolume-based pruning) and unbounded—that accumulate non-dominated solutions independently of the evolving population. This decouples the population-based search from the elite set, which is particularly useful for problems with more than two objectives [11]. Common quality indicators are built in, including hypervolume, generational distance (GD), inverted generational distance (IGD), and spread. Hypervolume computation can use optimized backends (MooCore) when installed, with fallback implementations for low-dimensional cases.

### H. Benchmark runner and reporting

The experiment layer includes a benchmark runner that executes standardized studies under a shared protocol (fixed problem definitions, evaluation budgets, and seeds) and emits machine-readable artifacts (per-seed results and summary CSVs). Table generation scripts produce LaTeX-ready tables directly from these artifacts, reducing transcription errors and enabling end-to-end reproducibility.

### I. Interactive dashboard and problem-definition tooling

The UX layer provides *VAMOS Studio*, a Streamlit-based dashboard designed to make multi-objective optimization accessible to users with no programming experience. It offers three main views: (i) a **Welcome** tab with a first-launch onboarding wizard and quick-reference tables for the CLI and Python API; (ii) a **Problem Builder** that lets users write objective and constraint functions in the browser, adjust bounds and algorithm settings, and see the Pareto front update live after each preview run; and (iii) an **Explore Results** tab for loading completed studies, comparing algorithms, and ranking solutions with multi-criteria decision-making (MCDM) methods. The Problem Builder ships with domain-specific templates (engineering design, machine learning, and scheduling) so that domain experts—e.g., a biologist optimizing a drug-candidate pipeline or a physicist fitting a multi-response

model—can start from a realistic example rather than a blank editor. Generated problems can be exported as standalone Python scripts or run directly through the `make_problem` convenience API (Listing 3).

## IV. Experimental Evaluation

We evaluate VAMOS along two dimensions: runtime efficiency across backends and frameworks, and solution quality measured by normalized hypervolume under a fixed-reference-front protocol. We first describe the benchmark suite and the semantic-alignment procedure that ensures cross-framework fairness, then report backend-level runtime comparisons and a scalability study of the vectorized architecture, followed by cross-framework runtime comparisons, solution-quality summaries, and statistical analyses.

### A. Benchmark suite and configuration alignment

We evaluate runtime and solution quality on widely used synthetic benchmarks: ZDT1–4 and ZDT6 [33] (two objectives; ZDT5 is excluded because it uses a binary encoding), DTLZ1–7 [34] (three objectives), and WFG1–9 [35] (two objectives). Problem dimensionalities are fixed to standard definitions: for example, DTLZ2/3/4 use $n = m + k - 1$ with $m = 3$ and $k = 10$ (thus $n = 12$). Non-standard DTLZ dimensionalities are permitted but explicitly flagged to avoid accidental comparisons against canonical settings. Unless otherwise stated, each (problem, framework) configuration is executed for 50,000 objective evaluations with 30 independent seeds.

Fig. 3 summarizes the benchmark protocol and semantic-alignment workflow used to ensure cross-framework fairness and to support the "same quality, faster" claim.

Although VAMOS implements nine MOEAs, our cross-framework comparison focuses on algorithms with the closest semantic matches across libraries: NSGA-II (baseline plus steady-state and archive variants), SMS-EMOA, and MOEA/D. This scope avoids confounding effects from missing implementations or materially different operator and termination semantics in specific frameworks. We restrict the comparison to pymoo and jMetalPy, the two most actively maintained and feature-rich Python MOEA libraries. DEAP and Platypus are excluded from the experimental evaluation: both offer limited algorithmic configurability (e.g., no native support for steady-state mode, external archives, or SMS-EMOA), lack modern APIs for variant configuration, are significantly slower than array-oriented frameworks, and have seen reduced maintenance activity in recent years.

To ensure comparability across frameworks, we standardize the algorithmic settings of NSGA-II: population size $N = 100$, SBX crossover probability $p_c = 1.0$ and distribution index $\eta_c = 20$, polynomial mutation distribution index $\eta_m = 20$ and mutation probability $p_m = 1/n$, tournament selection, and rank–crowding survival. For each framework we map these settings to the closest available implementation.

For SMS-EMOA we follow the same protocol and align what we can: population size $N = 100$, SBX ($p_c = 1.0$, $\eta_c = 20$), polynomial mutation ($p_m = 1/n$, $\eta_m = 20$), random parent selection, and a steady-state $(\mu + 1)$ loop (one offspring per iteration). However, the hypervolume-contribution reference point is not defined consistently across Frameworks: VAMOS and jMetalPy compute contributions in raw objective space with an adaptive reference point $r = \max(F)+1$, while pymoo normalizes objectives and uses a fixed shifted reference point $r = \mathbf{1} + \epsilon$ (we set $\epsilon = 1$). We therefore report the cross-framework SMS-EMOA comparison with this caveat.

For MOEA/D we align population size $N = 100$, weight vectors (uniform for two objectives; a shared $W3D\_100$ set for three objectives), neighborhood size $T = 20$, neighborhood selection probability $\delta = 0.9$, and replacement limit $\eta = 20$ (effectively unrestricted, matching pymoo's update rule). We use penalty-based boundary intersection (PBI) scalarization with $\theta = 5$ and a differential evolution (DE)/current/1/bin-style crossover ($CR = 1.0$, $F = 0.5$) followed by polynomial mutation ($p_m = 1/n$, $\eta_m = 20$), mapping these settings to the closest available implementation in each toolkit.

This workflow follows standard practice in comparative MOEA studies (independent runs, indicator computation, and structured reporting) and is consistent with the experimentation methodology described in jMetalPy [7]. Our main extension is to make the *inputs* to this workflow comparable across Frameworks by enforcing semantic alignment of both problem definitions and operator probabilities.

Crucially, we align *semantics* (not only parameter names) and enforce consistent benchmark definitions. For example, in **pymoo** the polynomial mutation operator exposes both an individual-level probability (`prob`) and a per-variable probability (`prob_var`); to match the standard "$p_m = 1/n$ per decision variable" setting used by jMetalPy, we set `prob`=1 and `prob_var`=1/n.

For WFG we adopt pymoo's canonical parameterization for two objectives ($k = 4$, $l = 20$ for $n = 24$) and pass parameters explicitly when a toolkit exposes different defaults. While full equivalence cannot be guaranteed due to framework-specific details (tie-breaking, boundary handling, duplicate elimination, etc.), these choices aim to make the comparison as fair as possible by removing confounders unrelated to algorithmic overhead and numerical kernels.

As an additional validity check, we sample 64 random decision vectors per problem uniformly within the specified bounds and verify that objective values match across implementations within a relative tolerance of $10^{-6}$ and an absolute tolerance of $10^{-8}$. This guards against silent benchmark-definition drift (e.g., differing domains or parameterizations) that could invalidate hypervolume comparisons.

### B. Runtime measurement

All runtimes are measured wall-clock using high-resolution timers and include algorithm overhead and objective evaluations. Unless otherwise stated, we use a fixed budget of 50,000 objective evaluations per run and report medians across 30 independent seeds, with per-problem and per-family summaries.

For Numba-accelerated backends, we distinguish two timing policies. **Warm** timings exclude one-time JIT compilation by performing a short warmup run in the same process before

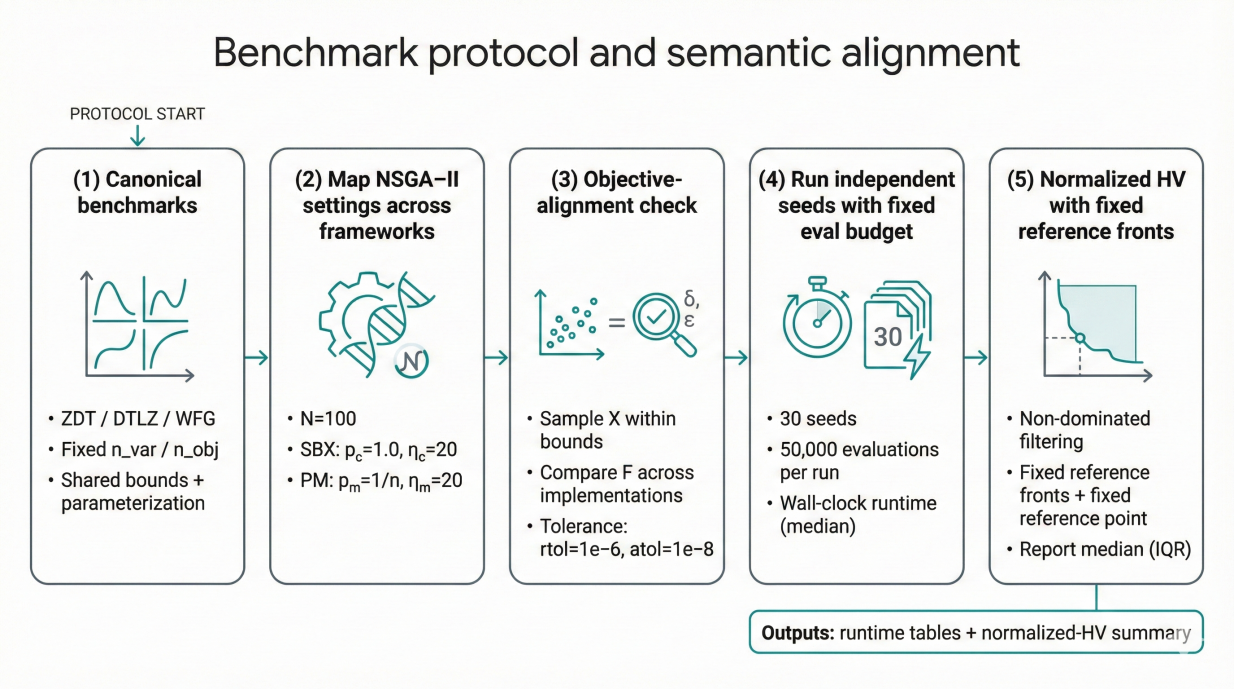


Fig. 3. Benchmark protocol and semantic alignment: map equivalent NSGA-II settings across frameworks, validate objective definitions by sampling decision vectors within bounds, run independent seeds under a fixed evaluation budget, and compute normalized hypervolume using fixed reference fronts.

starting the timer (2,000 warmup evaluations; not counted in the reported time). **Cold** timings measure the first run from a fresh process and therefore include JIT compilation overhead. Unless otherwise stated, we report warm timings to reflect steady-state throughput; cold-start costs can be reproduced with the provided scripts.

Experiments were run on a workstation with an Intel(R) Core(TM) Ultra 9 185H central processing unit (CPU) (16 cores, 22 logical processors) and 32 GB random-access memory (RAM), running Microsoft Windows 11 Home (build 26200). We used Python 3.12.3 and evaluated VAMOS 0.1.0 (NumPy 2.3.5 with OpenBLAS 0.3.30; Numba 0.63.1) against pymoo 0.6.1.6 and jMetalPy.

### C. Normalized hypervolume protocol

Hypervolume (HV), denoted $\mathrm{HV}(A, r)$, measures the Lebesgue measure of the region dominated by an approximation set $A$ with respect to a reference point $r$ (for minimization) [36]. Because HV is scale-dependent and sensitive to the choice of $r$, naive implementations can produce values that are not comparable across runs (e.g., if $r$ is expanded per run) and can be misleading when dominated solutions are included. To ensure a stable and comparable quality metric, we adopt the following protocol:

1) **Non-dominated filtering**: for every framework we compute HV on the final non-dominated set only.
2) **Fixed reference Pareto fronts**: for each benchmark problem we store a dense reference Pareto front $P_{\text{ref}}$. For ZDT and DTLZ (except DTLZ7) these fronts are generated analytically; for DTLZ7 and WFG we generate $P_{\text{ref}}$ by dense sampling followed by non-dominated filtering.
3) **Fixed reference point**: we set $r = \max(P_{\text{ref}}) + \epsilon$ (component-wise) with a small $\epsilon$, and we disallow run-dependent expansion.
4) **Normalization**: we report $\mathrm{HV}(A, r)/\mathrm{HV}(P_{\text{ref}}, r)$, yielding a dimensionless score typically in $[0, 1]$ and reducing sensitivity to objective scaling.

For WFG problems, the reference front is necessarily an approximation; we use large Pareto-set samples and non-dominated filtering, and we increase density for particularly sensitive cases (e.g., WFG2) to avoid underestimating $\mathrm{HV}(P_{\text{ref}}, r)$ and obtaining normalized HV slightly above 1.

As a complementary quality indicator, we also report the Inverted Generational Distance Plus (IGD+) [37], which measures the average minimum distance from each reference-front point to the closest solution in the approximation set (lower is better). Unlike HV, IGD+ captures both proximity and diversity with respect to the entire reference front. We report IGD+ summaries alongside normalized HV in Section IV-I.

### D. VAMOS backend comparison

Table III reports median runtime for VAMOS backends aggregated by problem family.

TABLE III
VAMOS BACKEND COMPARISON: MEDIAN RUNTIME (SECONDS) BY PROBLEM FAMILY

| Backend | ZDT | DTLZ | WFG | Average |
|---|---|---|---|---|
| Numba | **0.88** | **0.91** | **4.19** | **1.99** |
| MooCore | 0.91 | 0.92 | 7.06 | 2.96 |
| NumPy | 2.09 | 2.11 | 11.04 | 5.08 |

### E. Scalability of vectorized acceleration

To quantify how the array-native architecture scales, we measure wall-clock runtime of the Numba and NumPy backends as a function of population size on three representative problems (ZDT4, DTLZ2, WFG2) using NSGA-II with 50,000 evaluations and 5 independent seeds per configuration. Population sizes range from 50 to 800; all other settings match the main benchmark protocol.

Fig. 4 shows that the runtime gap between NumPy and Numba widens substantially as the population grows. At $N = 50$ the two backends perform similarly (and NumPy is even marginally faster on ZDT4), but by $N = 800$ the NumPy backend requires 30 s on all three problems while Numba stays below 12 s—a 3–5× speedup. The effect is consistent across problem families despite their different dimensionalities ($n_{\text{var}} = 10$, 12, and 24 for ZDT4, DTLZ2, and WFG2 respectively). Crucially, the Numba curve remains nearly flat across population sizes, confirming that the JIT-compiled kernels scale sub-linearly with array size in the per-evaluation cost, whereas NumPy's interpreted dispatch overhead accumulates with each generation cycle. These results reinforce the practical relevance of the vectorized design for large-scale benchmarking studies that require large populations or many independent seeds under a fixed compute budget.

### F. Convergence analysis

To verify that runtime speedups do not come at the expense of convergence rate, we track normalized hypervolume at 1,000-evaluation intervals over the full 50,000-evaluation budget. Fig. 5 compares VAMOS and pymoo on two representative problems: DTLZ2 (3 objectives, 12 variables) and ZDT1 (2 objectives, 30 variables). The convergence curves are nearly indistinguishable, with overlapping interquartile ranges across all 30 seeds. Both frameworks reach the same final quality at the same evaluation counts, confirming that the performance gains observed in runtime measurements are not accompanied by degraded convergence behavior.

### G. Pareto front visualization

Fig. 6 provides a visual comparison of the final non-dominated sets produced by VAMOS, pymoo, and jMetalPy on ZDT1 (2D Pareto front) and DTLZ2 (3D Pareto front). In both cases, the solutions from all three frameworks overlap closely with the reference front and with each other, providing qualitative confirmation of the quantitative equivalence reported in the hypervolume and IGD+ summaries. The 3D DTLZ2 plot illustrates that VAMOS's approximation set covers the spherical Pareto surface with comparable uniformity to the established frameworks.

### H. Cross-framework comparison

Fig. 7 compares VAMOS against other Python frameworks on runtime. VAMOS is evaluated with its accelerated backend (Numba) for the headline comparison, while additional backends are reported in the appendix. We repeat the same runtime protocol for NSGA-II variants (steady-state and external archive), SMS-EMOA (steady-state, one offspring per iteration), and MOEA/D (decomposition with differential-evolution variation), reported in Table VII (Appendix A).

Fig. 7 visualizes relative runtimes. Each bar represents the ratio of a competitor's median family-level runtime to VAMOS (Numba); the dashed line at $2^0 = 1$ marks parity. jMetalPy is consistently 3–11× slower, with the largest gaps on WFG problems where per-solution overhead is amplified by higher dimensionality ($n = 24$). pymoo is the closest competitor, staying within 1.1–1.2× of VAMOS on ZDT and DTLZ, though the gap widens on WFG. Error bars indicate variability across per-problem medians within each family.

*a) Steady-state NSGA-II.:* Some Frameworks expose a steady-state (incremental-replacement) variant of NSGA-II, typically implemented as a $(\mu + 1)$ loop with one offspring per iteration and one survivor replaced. We benchmark this mode by aligning NSGA-II settings across frameworks and setting the offspring batch size to 1. In VAMOS, we configure steady-state mode with `offspring_size=1` and `replacement_size=1`; in pymoo we set `n_offsprings=1`; in jMetalPy we set `offspring_population_size=1`.

*b) NSGA-II with external archive.:* To assess the overhead of maintaining an external elite set, we also benchmark NSGA-II with an *unbounded* external archive that accumulates non-dominated solutions throughout the run. For this variant, we compute normalized hypervolume on the archive contents (rather than the final population) to reflect the larger retained approximation set. In VAMOS, we enable an external archive with `unbounded=True` and size initialized to the population size; in pymoo we maintain a separate unbounded archive updated from the evolving population/offspring; in jMetalPy we use an NSGA-II implementation with an archive hook.

*c) Memory footprint.:* To complement the runtime comparison, we measure peak resident-set-size (RSS) overhead during NSGA-II runs on three representative problems (ZDT1, DTLZ2, WFG4) using 50,000 evaluations and 5 seeds per framework (Fig. 8). VAMOS achieves a median peak memory of 0.28 MB, compared to 0.84 MB for pymoo (3× higher) and 0.50 MB for jMetalPy (1.8× higher). The lower memory footprint reflects the array-native representation, which stores the entire population in contiguous NumPy arrays rather than materializing individual solution objects with per-instance attribute dictionaries.

### I. Solution quality summary

Table IV summarizes normalized hypervolume for the NSGA-II baseline across frameworks, aggregated by benchmark family. We report per-problem medians across seeds, and summarize these per-problem medians using median and interquartile range (IQR) within each family, yielding a descriptive view of solution quality under the fixed normalization protocol (Section IV-C).

Across ZDT, all frameworks attain nearly identical normalized HV, suggesting that benchmark definitions and operator

[0] Average* in summary tables denotes the mean of family-level medians (3 values). Per-problem runtimes are reported in Fig. 10.

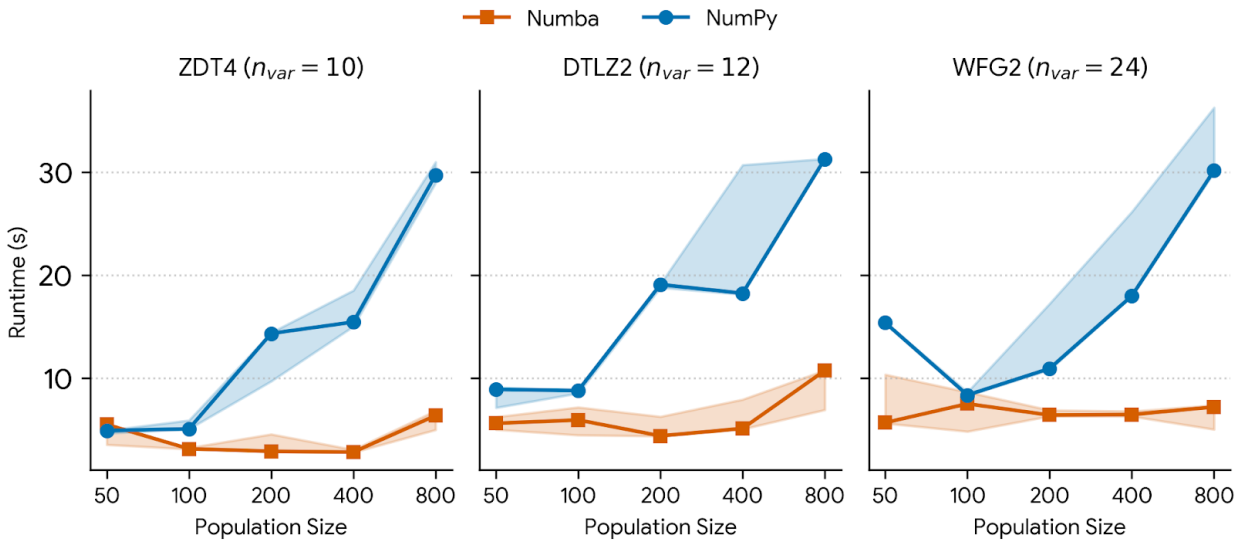


Fig. 4. Runtime scaling of VAMOS backends (Numba vs. NumPy) as a function of population size on three representative problems. Shaded regions indicate the interquartile range across 5 seeds. NumPy runtime grows steeply with population size due to increasing Python-level overhead in per-generation operations, while Numba runtime remains nearly flat, demonstrating that JIT-compiled kernels amortize loop overhead effectively on larger arrays.

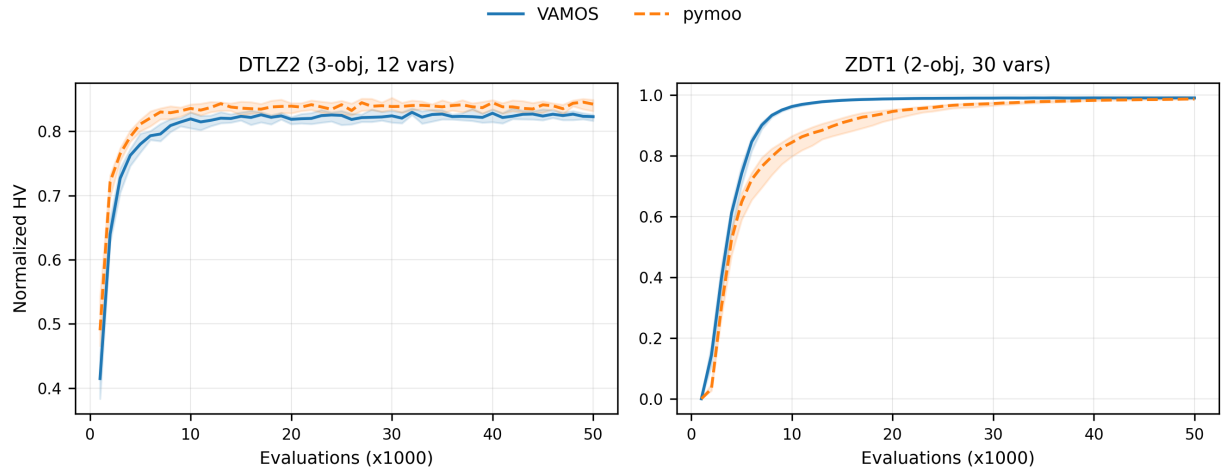


Fig. 5. Convergence of normalized hypervolume over 50,000 evaluations for VAMOS (Numba) and pymoo on DTLZ2 (3 objectives, 12 variables) and ZDT1 (2 objectives, 30 variables). Solid/dashed lines show median HV across 30 seeds; shaded regions indicate the interquartile range. Both frameworks follow nearly identical convergence trajectories, confirming that the semantic alignment protocol produces equivalent algorithmic behavior.

TABLE IV
NORMALIZED HYPERVOLUME SUMMARY (MEDIAN (IQR)) BY PROBLEM FAMILY ACROSS FRAMEWORKS

| Family | VAMOS | pymoo | jMetalPy |
|---|---|---|---|
| ZDT | **0.991 (0.013)** | **0.991 (0.011)** | **0.991 (0.014)** |
| DTLZ | **0.836 (0.121)** | 0.829 (0.127) | 0.834 (0.124) |
| WFG | **0.934 (0.138)** | 0.846 (0.220) | 0.843 (0.218) |
| Overall | **0.934 (0.157)** | 0.918 (0.156) | 0.922 (0.158) |

semantics are effectively aligned. For DTLZ, medians are tightly clustered across frameworks, while WFG exhibits the largest dispersion; VAMOS achieves the highest median on WFG and overall. Together with the runtime results, these summaries support improved throughput without degraded final quality under the fixed-reference-front protocol.

Table V extends this analysis to the NSGA-II steady-state and external-archive variants. For the steady-state variant, all three frameworks achieve nearly identical overall HV. For the archive variant, all three frameworks reach comparable normalized HV across ZDT and DTLZ families (1.000), confirming that the unbounded archive accumulates equivalent approximation sets regardless of the framework.

As a complementary quality indicator, Table VI reports IGD+ summaries across frameworks for NSGA-II, SMS-EMOA, and MOEA/D. For NSGA-II, IGD+ values are tightly clustered across frameworks (overall median 0.0108 for VAMOS and pymoo, 0.0105 for jMetalPy), mirroring the HV equivalence. SMS-EMOA achieves the lowest IGD+ overall (0.0078 for all three frameworks), reflecting the steady-state indicator-based selection's effectiveness at maintaining proximity and diversity. For MOEA/D, VAMOS (0.0102) and

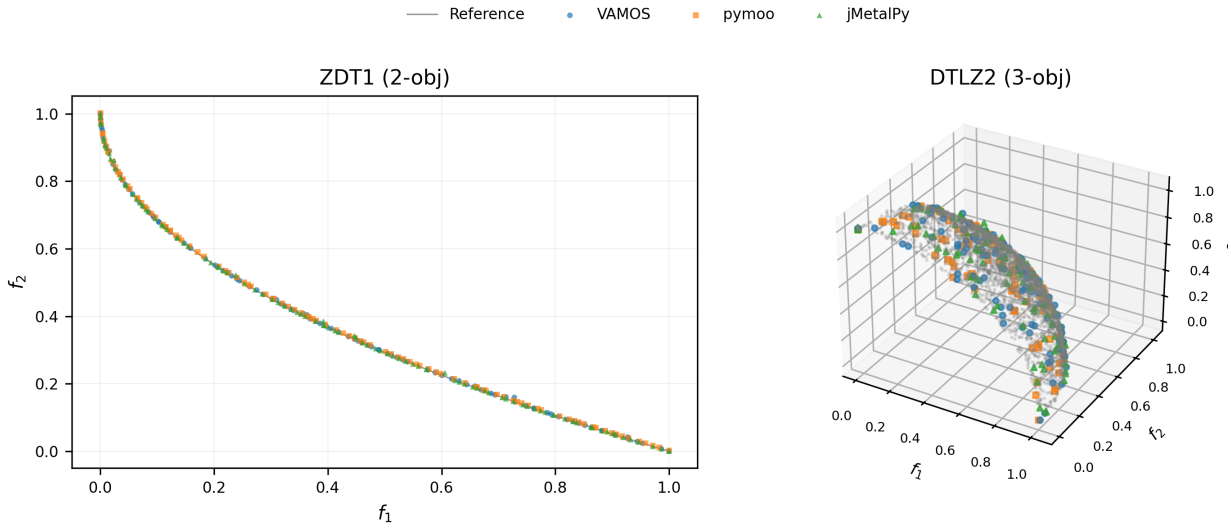


Fig. 6. Final non-dominated sets from VAMOS, pymoo, and jMetalPy on ZDT1 (left, 2-objective) and DTLZ2 (right, 3-objective), overlaid with the reference Pareto front. All three frameworks produce well-distributed approximation sets that closely track the reference front, confirming solution-quality equivalence under the fixed-reference-front protocol.

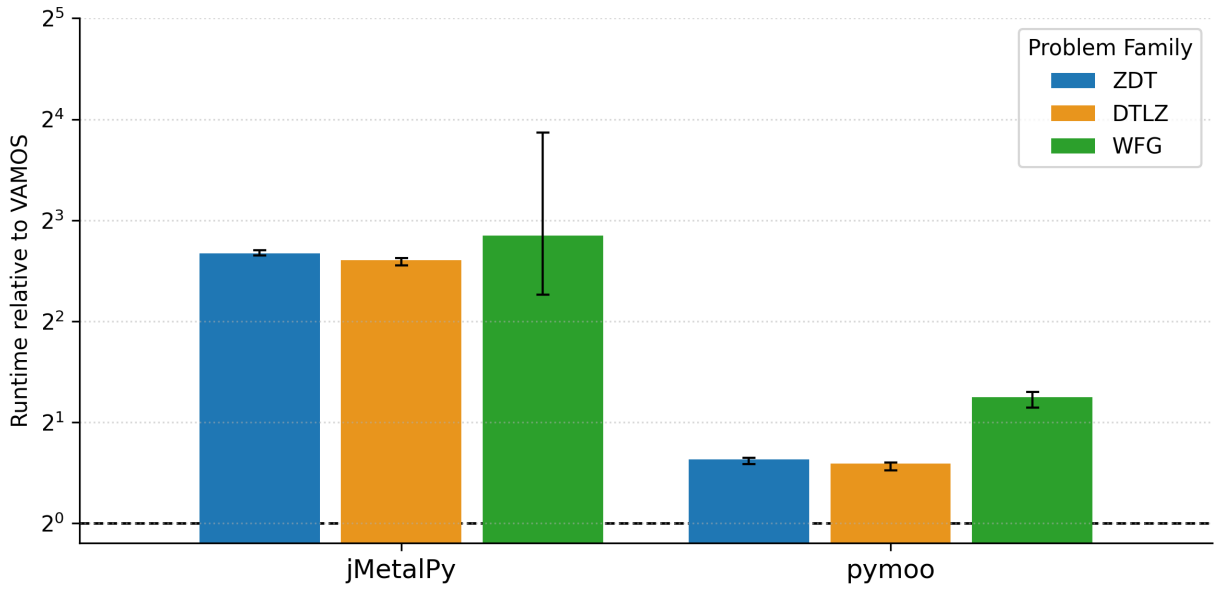


Fig. 7. Runtime of each framework relative to VAMOS (Numba) for generational NSGA-II, grouped by problem family ($\log_2$ scale). Bars above the dashed parity line ($2^0 = 1$) indicate that the competitor is slower. Error bars show variability across per-problem medians within each family. All runs use 50,000 evaluations and 30 seeds.

TABLE V
NORMALIZED HYPERVOLUME SUMMARY (MEDIAN (IQR)) BY PROBLEM FAMILY FOR NSGA-II VARIANTS

| **Algorithm** | **Framework** | **ZDT** | **DTLZ** | **WFG** | **Overall*** |
|---|---|---|---|---|---|
| NSGA-II (steady-state) | VAMOS | 0.993 (0.009) | 0.843 (0.096) | 0.930 (0.138) | 0.930 (0.153) |
| | pymoo | 0.993 (0.008) | 0.847 (0.122) | 0.849 (0.218) | 0.921 (0.153) |
| | jMetalPy | 0.993 (0.009) | 0.846 (0.098) | 0.844 (0.218) | 0.919 (0.153) |
| NSGA-II (ext. archive) | VAMOS | 1.000 (0.000) | 1.000 (0.078) | 0.959 (0.138) | 0.997 (0.139) |
| | pymoo | 1.000 (0.000) | 1.001 (0.093) | 0.868 (0.224) | 0.996 (0.152) |
| | jMetalPy | 1.000 (0.000) | 1.001 (0.097) | 0.960 (0.138) | 0.998 (0.129) |

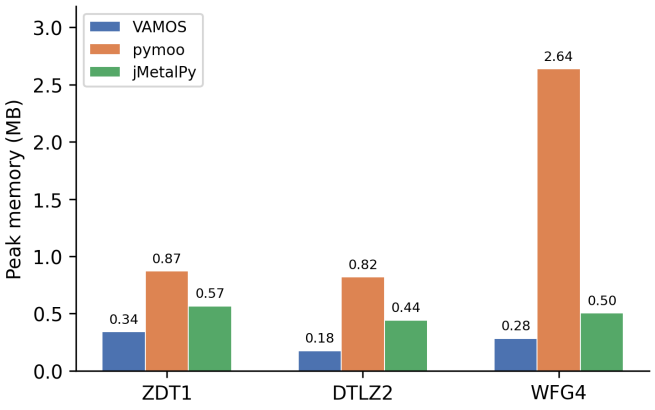


Fig. 8. Peak memory footprint (MB) during NSGA-II optimization on three representative problems (50,000 evaluations, median of 5 seeds). VAMOS consistently uses less memory than both pymoo and jMetalPy across all problem families.

jMetalPy (0.0102) outperform pymoo (0.0160), consistent with the known difficulty of MOEA/D under certain weight-vector configurations.

### J. Statistical analysis

To move beyond descriptive summaries, we apply Wilcoxon signed-rank tests (paired by seed) comparing VAMOS against each competitor on every problem, with Holm step-down correction for familywise error control ($\alpha = 0.05$). We report the Vargha–Delaney $\hat{A}_{12}$ effect size [38] alongside each test ($\hat{A}_{12} > 0.5$ favors VAMOS; thresholds: negligible $< 0.56$, small $< 0.64$, medium $< 0.71$, large$\geq$0.71). We also report paired-bootstrap 90% confidence intervals for the relative normalized-HV difference $\Delta = (\mathrm{HV}_{\mathrm{VAMOS}} - \mathrm{HV}_{\mathrm{fw}})/\mathrm{HV}_{\mathrm{fw}}$, and we declare *equivalence* when the entire CI falls within $\pm 1\%$.

Table VIII presents the per-problem Wilcoxon results for the NSGA-II baseline. Of 21 problems, only one (ZDT6) shows a statistically significant difference in normalized HV after Holm correction, and the magnitude is small ($\Delta \approx 0.3\%$). Table X summarizes the equivalence analysis: VAMOS is equivalent to pymoo on 14/21 problems and non-inferior on 16/21, with a median $\Delta$HV of $-0.03\%$. Similar patterns hold for jMetalPy. Table XI details robustness, and Fig. 9 visualizes per-problem CIs as a forest plot. Corresponding analyses for SMS-EMOA and MOEA/D are provided in Appendix D. Detailed NSGA-II statistical tables are provided in Appendix B.

### K. Discussion

The observed speedups are primarily due to (i) the array-based representation that removes per-solution Python overhead, and (ii) JIT compilation of hot operators and utilities in the Numba backend. The scalability study (Fig. 4) confirms this mechanism: the NumPy backend's runtime grows steeply with population size, reflecting per-generation interpreted overhead, while the Numba backend remains nearly flat. Table IV summarizes that these performance gains are not obtained at the expense of final solution quality under the fixed-reference-front normalized-HV protocol, avoiding artifacts caused by run-dependent reference points.

As Fig. 7 illustrates, the cross-framework gaps are problem-family dependent: WFG problems exhibit the largest relative slowdowns for object-centric libraries, consistent with their higher dimensionality amplifying per-solution overhead. The magnitude of the speedup also varies across algorithm variants in a pattern consistent with this overhead model. Steady-state variants (Table VII) show the largest relative speedups because they perform one selection–variation–survival cycle per offspring: frameworks with high per-iteration Python overhead incur that cost 50,000 times (one per evaluation) rather than 500 times (one per generation of 100 offspring). VAMOS's array-kernel dispatch amortizes this overhead more effectively, widening the gap.

Not all comparisons favor VAMOS. For MOEA/D on WFG problems (Table VII), jMetalPy achieves lower median runtime (8.34 s vs. 11.12 s). This is attributable to differences in the decomposition inner loop: jMetalPy's MOEA/D processes one subproblem at a time with lightweight scalar operations, a pattern where Python-level overhead is modest and VAMOS's array-batching strategy does not yield the same advantage as in generational algorithms with population-wide survival.

Beyond runtime, VAMOS offers a level of component-level configurability that distinguishes it from the closest competitor. pymoo provides a modular API for standard algorithm configurations, but does not natively support configuring the offspring batch size to obtain steady-state variants, attaching bounded or unbounded external archives to arbitrary algorithms, switching between multiple constraint-handling modes (feasibility rules, penalty functions, or $\varepsilon$-relaxation). In contrast, VAMOS exposes all of these as first-class configuration options—as demonstrated experimentally by the steady-state NSGA-II and external-archive variants (Table VII), which required only parameter changes in VAMOS but custom wrappers or workarounds in other Frameworks. This configurability is complemented by the ability to swap the compute backend (NumPy, Numba, MooCore) with a single parameter, enabling users to trade compilation overhead for throughput depending on their use case.

The third design axis—accessibility—addresses a different audience than the first two. While performance and configurability primarily benefit MOEA researchers running large-scale benchmarking studies, the minimal-boilerplate API and graphical tooling target domain experts who need multi-objective optimization but lack a background in metaheuristics. The two-line `optimize` entry point, the `make_problem` factory, and the `algorithm="auto"` mode eliminate the need to understand algorithm internals, operator semantics, or framework-specific abstractions. VAMOS Studio further lowers the barrier by providing a browser-based environment with domain-specific templates where users can define problems, run optimizations, and explore Pareto fronts without writing any code. As Table I shows, no other Python MOEA framework combines all three axes: array-native performance, rich configurability, and a two-line API with a graphical interface. We note that usability is assessed here through objective proxies (lines of code, number of required imports,

TABLE VI
IGD+ SUMMARY (MEDIAN (IQR)) BY PROBLEM FAMILY ACROSS FRAMEWORKS AND ALGORITHMS. LOWER VALUES INDICATE CLOSER PROXIMITY TO THE REFERENCE PARETO FRONT

| Algorithm | Framework | ZDT | DTLZ | WFG | Overall* |
|---|---|---|---|---|---|
| NSGA-II | VAMOS | 0.0028 (0.0015) | 0.0352 (0.0200) | 0.0626 (0.1286) | 0.0257 (0.0594) |
| | pymoo | 0.0025 (0.0015) | 0.0351 (0.0216) | 0.0700 (0.1294) | 0.0350 (0.0669) |
| | jMetalPy | 0.0030 (0.0015) | 0.0351 (0.0219) | 0.0698 (0.1276) | 0.0347 (0.0667) |
| SMS-EMOA | VAMOS | 0.0019 (0.0012) | 0.0186 (0.0079) | 0.0560 (0.1275) | 0.0179 (0.0537) |
| | pymoo | 0.0019 (0.0012) | 0.0217 (0.0393) | 0.0552 (0.1273) | 0.0184 (0.0658) |
| | jMetalPy | 0.0018 (0.0012) | 0.0221 (0.0318) | 0.0546 (0.1281) | 0.0183 (0.0659) |
| MOEA/D | VAMOS | 0.0060 (0.0053) | 0.0421 (0.0464) | 0.1252 (0.1203) | 0.0421 (0.1154) |
| | pymoo | 0.0106 (0.0033) | 0.0464 (0.0532) | 0.1103 (0.1308) | 0.0560 (0.0964) |
| | jMetalPy | 0.0057 (0.0054) | 0.0438 (0.0481) | 0.1229 (0.1272) | 0.0438 (0.1132) |

and feature availability) rather than controlled user studies; a formal evaluation with domain-expert participants is left to future work.

These speedups have direct practical implications. A 4–10× reduction in per-run wall-clock time means that the same compute budget can accommodate proportionally more independent seeds, larger population sizes, or denser hyperparameter sweeps—precisely the conditions required for statistically rigorous benchmarking studies. Moreover, the scaling study (Section IV-E) shows that the advantage grows with population size, so the benefit is amplified in exactly the large-scale regimes where it matters most.

We also note that both VAMOS and pymoo obtain zero normalized hypervolume on DTLZ3 under MOEA/D with 50,000 evaluations. This is a known difficulty: DTLZ3 is a multimodal problem designed to test convergence, and MOEA/D with PBI scalarization and the evaluation budget used here is insufficient to escape local fronts consistently. This outcome is not framework-specific but reflects the interaction between algorithm configuration and problem difficulty.

A related observation concerns SMS-EMOA on DTLZ4 (Appendix D): VAMOS attains a median normalized HV of 0.898 versus 0.671 for pymoo, yet the Wilcoxon test reports $p = 1.0$ after Holm correction. This reflects very high variance across seeds due to the deceptive density bias in DTLZ4, which causes many seeds to converge to a narrow region of the front. The high IQR makes the paired differences too noisy for the test to reject the null hypothesis despite the large median gap.

The convergence analysis (Fig. 5) and Pareto front visualization (Fig. 6) reinforce the solution-quality equivalence: VAMOS and pymoo follow nearly identical convergence trajectories on both ZDT1 and DTLZ2, and the final nondominated sets from all three frameworks overlap closely with the reference fronts. These visual confirmations complement the statistical analyses and provide intuitive evidence that the runtime speedups do not degrade convergence behavior.

The memory footprint measurements (Fig. 8) further illustrate the practical advantages of the array-native design: VAMOS uses 3× less peak memory than pymoo and 1.8× less than jMetalPy on the same problems. For large-scale studies involving many concurrent runs or high-dimensional problems, this reduced memory overhead can be a meaningful practical benefit.

### L. Threats to validity

Despite careful alignment of dimensions, budgets, problem definitions, and operator settings, cross-framework comparisons remain subject to several threats: (i) operator implementations may differ in subtle details (e.g., boundary handling, duplicate elimination, tie-breaking in survival), (ii) random number generation and seeding semantics vary across libraries, and (iii) library-specific overheads (data conversion, object materialization) may affect measured runtime. We mitigate major sources of unfairness by enforcing consistent benchmark bounds (e.g., ZDT4) and matching operator semantics where APIs differ (e.g., mutation probability in pymoo).

Our cross-framework comparison is deliberately scoped to Python MOEA libraries whose inner loops are predominantly executed in Python. Libraries such as PyGMO (Python bindings to the C++ PaGMO2 library) execute algorithms and benchmark functions fully in compiled code, yielding fundamentally different runtime profiles; we therefore treat them as an orthogonal C++ baseline and leave their inclusion to future work with an explicit language/runtime study design.

For hypervolume, reference fronts for problems without closed-form Pareto fronts are necessarily approximations; while we generate them densely and fix them across runs, remaining discrepancies can affect absolute normalized values, especially on WFG problems. We therefore treat normalized HV primarily as a comparative metric under a fixed protocol, and we report distributions across multiple seeds rather than relying on single-run outcomes.

## V. CONCLUSION

We presented VAMOS, a high-performance Python framework for multi-objective optimization studies that addresses three practical gaps in existing Frameworks: (i) a vectorized core with pluggable compute kernels that reduces inner-loop overhead; (ii) rich component-level configurability—including interchangeable variation operators, configurable offspring batch size for steady-state variants, optional bounded or unbounded external archives, multiple constraint-handling modes—that enables researchers to explore algorithmic design choices without custom code; and (iii) an accessible API and

graphical interface that lowers the entry barrier for domain experts with basic Python skills, requiring as few as two lines of code for a complete optimization run. We also release a reproducible cross-framework benchmarking pipeline with semantic alignment and fixed reference Pareto fronts for normalized hypervolume reporting. Under this protocol, VAMOS achieves competitive normalized hypervolume (Table IV) while substantially reducing runtime (Fig. 7, Table VII). Convergence analysis (Fig. 5) and Pareto front visualization (Fig. 6) confirm that these speedups do not degrade convergence behavior or solution quality. The framework achieves $3\times$ lower peak memory than pymoo.

### A. Future work

- **Many-objective benchmarking**: extend the experimental protocol beyond $m = 3$ with scalable indicators.
- **Distributed GPU execution**: explore integration with EvoX [13] for multi-node GPU acceleration.
- **Algorithm configuration**: extend the tuning module with interoperability with external configurators such as irace [18], and richer tuning benchmarks and reports.
- **Broader comparisons**: extend the benchmark suite to additional MOEAs and frameworks and include time-to-target and other indicators alongside normalized HV.
- **LLM-assisted experiment planning**: we are exploring an integrated assistant that maps a natural-language problem description to a validated experiment configuration, using an optional LLM provider, to further lower the barrier to entry for non-expert users.
- **User-friendliness evaluation**: conduct controlled user studies with domain experts from fields such as engineering, biology, and operations research to formally assess the usability of the API, the graphical interface, and the documentation, and to identify concrete improvements that further reduce the learning curve for non-specialist users.

## Data and Code Availability

All data and code supporting the findings of this study (benchmark scripts, reference Pareto fronts, and CSV artifacts used to regenerate tables) are available in the VAMOS repository:
https://github.com/vamos-framework/vamos.

## Appendix A
## Additional Runtime Comparisons

Tables in this appendix report supplementary runtime comparisons for NSGA-II variants (steady-state and external archive), SMS-EMOA, and MOEA/D under the same protocol as Section IV.

## Appendix B
## Detailed Statistical Analysis for NSGA-II

This appendix reports per-problem Wilcoxon signed-rank tests, bootstrap equivalence analyses, robustness summaries, and bootstrap confidence intervals for the NSGA-II baseline discussed in Section IV-J.

TABLE VII
MEDIAN RUNTIME (SECONDS) BY PROBLEM FAMILY FOR ALGORITHM VARIANTS

| Algorithm | Framework | ZDT | DTLZ | WFG | Average |
|---|---|---|---|---|---|
| NSGA-II (steady-state) | VAMOS | **8.97** | **9.98** | **93.43** | **37.46** |
| | pymoo | 38.84 | 44.36 | 360.96 | 148.06 |
| | jMetalPy | 95.51 | 103.32 | 159.09 | 119.31 |
| NSGA-II (ext. archive) | VAMOS | **13.64** | **39.87** | **15.52** | **23.01** |
| | pymoo | 28.92 | 81.54 | 32.35 | 47.61 |
| | jMetalPy | 18.10 | 44.05 | 46.94 | 36.36 |
| SMS-EMOA | VAMOS | **5.86** | **6.94** | **97.84** | **36.88** |
| | pymoo | 38.35 | 45.81 | 489.74 | 191.30 |
| | jMetalPy | 92.45 | 100.22 | 170.72 | 121.13 |
| MOEA/D | VAMOS | **3.25** | **4.05** | **74.08** | **27.13** |
| | pymoo | 14.76 | 19.24 | 132.32 | 55.44 |
| | jMetalPy | 7.63 | 8.17 | 92.70 | 36.17 |

TABLE VIII
NSGA-II. NORMALIZED HYPERVOLUME COMPARISON WITH WILCOXON SIGNED-RANK TEST RESULTS (HOLM-CORRECTED)

| Problem | VAMOS | pymoo | p-value | $\hat{A}_{12}$ | Sig. |
|---|---|---|---|---|---|
| zdt1 | 0.991 | **0.991** | 1.000 | 0.40 | |
| zdt2 | **0.983** | 0.982 | 1.000 | 0.47 | |
| zdt3 | 0.996 | **0.996** | 1.000 | 0.42 | |
| zdt4 | 1.001 | **1.001** | 1.000 | 0.44 | |
| zdt6 | 0.983 | **0.985** | ¡0.0001 | 0.00 | ✓ |
| dtlz1 | **0.922** | 0.918 | 1.000 | 0.57 | |
| dtlz2 | 0.823 | **0.827** | 0.186 | 0.31 | |
| dtlz3 | **0.728** | 0.711 | 1.000 | 0.59 | |
| dtlz4 | **0.836** | 0.829 | 1.000 | 0.60 | |
| dtlz5 | 0.966 | **0.966** | 1.000 | 0.46 | |
| dtlz6 | 0.488 | **0.554** | 1.000 | 0.39 | |
| dtlz7 | 0.871 | **0.874** | 1.000 | 0.43 | |
| wfg1 | 0.412 | **0.430** | 1.000 | 0.45 | |
| wfg2 | 1.007 | **1.007** | 1.000 | 0.42 | |
| wfg3 | 0.973 | **0.973** | 1.000 | 0.49 | |
| wfg4 | 0.956 | **0.957** | 1.000 | 0.39 | |
| wfg5 | 0.826 | **0.826** | 0.244 | 0.32 | |
| wfg6 | 0.833 | **0.846** | 1.000 | 0.43 | |
| wfg7 | 0.964 | **0.964** | 1.000 | 0.48 | |
| wfg8 | 0.744 | **0.745** | 1.000 | 0.48 | |
| wfg9 | **0.934** | 0.714 | 1.000 | 0.58 | |

## Appendix C
## Reference Fronts and Detailed Benchmark Results

The repository includes dense reference Pareto fronts for all benchmark problems used for normalized hypervolume computation. These fronts are generated analytically when closed forms are available (ZDT and most DTLZ problems) and by dense sampling followed by non-dominated filtering for cases where the Pareto front is disconnected or defined implicitly (DTLZ7 and WFG). For WFG2 we use a higher sampling density to stabilize HV normalization.

Fig. 10 presents per-problem median runtimes as a dual-panel heatmap. The left panel compares VAMOS backends (Numba, MooCore, NumPy); the right panel compares VAMOS against pymoo and jMetalPy. Color intensity indicates runtime on a logarithmic scale, with annotated values in each cell and bold values marking the fastest per problem.

TABLE IX
NSGA-II. IGD+ COMPARISON WITH WILCOXON SIGNED-RANK TEST RESULTS (HOLM-CORRECTED). LOWER IS BETTER

| Problem | VAMOS | pymoo | p-value | $\hat{A}_{12}$ | Sig. |
|---|---|---|---|---|---|
| zdt1 | 0.0032 | **0.0031** | 1.000 | 0.59 | |
| zdt2 | 0.0030 | **0.0030** | 1.000 | 0.52 | |
| zdt3 | 0.0015 | **0.0015** | 1.000 | 0.54 | |
| zdt4 | 0.0009 | **0.0009** | 1.000 | 0.55 | |
| zdt6 | 0.0028 | **0.0025** | ¡0.0001 | 1.00 | ✓ |
| dtlz1 | 0.0185 | **0.0185** | 1.000 | 0.51 | |
| dtlz2 | 0.0358 | **0.0351** | 1.000 | 0.58 | |
| dtlz3 | **0.0572** | 0.0615 | 1.000 | 0.39 | |
| dtlz4 | **0.0345** | 0.0351 | 1.000 | 0.42 | |
| dtlz5 | 0.0029 | **0.0029** | 1.000 | 0.53 | |
| dtlz6 | 0.0872 | **0.0750** | 1.000 | 0.60 | |
| dtlz7 | 0.0352 | **0.0350** | 1.000 | 0.57 | |
| wfg1 | **0.6840** | 0.6949 | 1.000 | 0.55 | |
| wfg2 | 0.2158 | **0.2155** | 1.000 | 0.58 | |
| wfg3 | 0.0191 | **0.0191** | 1.000 | 0.52 | |
| wfg4 | 0.0132 | **0.0128** | 1.000 | 0.61 | |
| wfg5 | **0.0699** | 0.0700 | 1.000 | 0.51 | |
| wfg6 | 0.0626 | **0.0573** | 1.000 | 0.58 | |
| wfg7 | 0.0104 | **0.0103** | 1.000 | 0.53 | |
| wfg8 | **0.1477** | 0.1485 | 1.000 | 0.49 | |
| wfg9 | **0.0257** | 0.1264 | 1.000 | 0.46 | |

TABLE X
NORMALIZED HYPERVOLUME EQUIVALENCE SUMMARY (PAIRED BOOTSTRAP 90% CI; EQUIVALENCE MARGIN $\pm 1\%$ RELATIVE)

| Framework | Eq. problems | Non-inf. problems | Median $\Delta$HV (%) |
|---|---|---|---|
| pymoo | 14/21 | 16/21 | -0.03 |
| jMetalPy | 16/21 | 16/21 | +0.01 |

## APPENDIX D
## STATISTICAL ANALYSIS FOR SMS-EMOA AND MOEA/D

Tables in this appendix report Wilcoxon signed-rank tests, bootstrap equivalence analyses, and robustness summaries for SMS-EMOA and MOEA/D, following the same methodology as Section IV-J.

### A. SMS-EMOA

Tables XII–XV present per-problem Wilcoxon tests, equivalence summaries, and robustness summaries for SMS-EMOA. Fig. 11 visualizes per-problem bootstrap confidence intervals.

### B. MOEA/D

Tables XVI–XIX and Fig. 12 present the corresponding analyses for MOEA/D.

TABLE XI
ROBUSTNESS SUMMARY FOR NORMALIZED HYPERVOLUME ACROSS SEEDS (MEDIAN ACROSS PROBLEMS). "NEAR-BEST" IS DEFINED PER PROBLEM AS $\mathrm{HV} \geq 0.99 \cdot \max \mathrm{MEDIAN(HV)}$ AMONG FRAMEWORKS

| Framework | Median IQR(HV) | Median % seeds near-best |
|---|---|---|
| VAMOS | 0.004 | 93.3 |
| pymoo | 0.006 | 96.7 |
| jMetalPy | 0.005 | 96.7 |

TABLE XII
SMS-EMOA. NORMALIZED HYPERVOLUME COMPARISON WITH WILCOXON SIGNED-RANK TEST RESULTS (HOLM-CORRECTED)

| Problem | VAMOS | pymoo | p-value | $\hat{A}_{12}$ | Sig. |
|---|---|---|---|---|---|
| zdt1 | 0.993 | **0.993** | 1.000 | 0.46 | |
| zdt2 | 0.987 | **0.987** | 1.000 | 0.43 | |
| zdt3 | **0.997** | 0.997 | 1.000 | 0.50 | |
| zdt4 | **1.004** | 1.004 | 1.000 | 0.57 | |
| zdt6 | **0.989** | 0.989 | 1.000 | 0.58 | |
| dtlz1 | **0.959** | 0.959 | 1.000 | 0.56 | |
| dtlz2 | **0.933** | 0.933 | 1.000 | 0.53 | |
| dtlz3 | **0.903** | 0.903 | 1.000 | 0.50 | |
| dtlz4 | **0.933** | 0.712 | 0.730 | 0.60 | |
| dtlz5 | 0.978 | **0.978** | 0.002 | 0.21 | ✓ |
| dtlz6 | 0.553 | **0.566** | 1.000 | 0.44 | |
| dtlz7 | 0.949 | **0.950** | 0.152 | 0.17 | |
| wfg1 | 0.244 | **0.258** | 1.000 | 0.46 | |
| wfg2 | 1.009 | **1.010** | 1.000 | 0.49 | |
| wfg3 | 0.983 | **0.983** | 1.000 | 0.43 | |
| wfg4 | **0.980** | 0.980 | 1.000 | 0.56 | |
| wfg5 | 0.836 | **0.836** | 1.000 | 0.52 | |
| wfg6 | 0.856 | **0.858** | 1.000 | 0.51 | |
| wfg7 | 0.982 | **0.982** | 1.000 | 0.45 | |
| wfg8 | 0.774 | **0.775** | 1.000 | 0.44 | |
| wfg9 | **0.963** | 0.961 | 1.000 | 0.53 | |

TABLE XIII
SMS-EMOA. IGD+ COMPARISON WITH WILCOXON SIGNED-RANK TEST RESULTS (HOLM-CORRECTED). LOWER IS BETTER

| Problem | VAMOS | pymoo | p-value | $\hat{A}_{12}$ | Sig. |
|---|---|---|---|---|---|
| zdt1 | 0.0023 | **0.0023** | 1.000 | 0.52 | |
| zdt2 | **0.0022** | 0.0022 | 1.000 | 0.52 | |
| zdt3 | **0.0010** | 0.0010 | 1.000 | 0.47 | |
| zdt4 | **0.0003** | 0.0003 | 1.000 | 0.47 | |
| zdt6 | **0.0019** | 0.0019 | 1.000 | 0.43 | |
| dtlz1 | **0.0134** | 0.0134 | 1.000 | 0.47 | |
| dtlz2 | **0.0183** | 0.0184 | 1.000 | 0.45 | |
| dtlz3 | 0.0254 | **0.0252** | 1.000 | 0.51 | |
| dtlz4 | **0.0186** | 0.1090 | 1.000 | 0.50 | |
| dtlz5 | 0.0017 | **0.0017** | 1.000 | 0.57 | |
| dtlz6 | 0.0877 | **0.0852** | 1.000 | 0.56 | |
| dtlz7 | 0.0221 | **0.0217** | 0.305 | 0.79 | |
| wfg1 | 0.8724 | **0.8573** | 1.000 | 0.53 | |
| wfg2 | 0.2140 | **0.2138** | 1.000 | 0.52 | |
| wfg3 | 0.0118 | **0.0115** | 1.000 | 0.57 | |
| wfg4 | **0.0062** | 0.0062 | 1.000 | 0.51 | |
| wfg5 | 0.0682 | **0.0681** | 1.000 | 0.63 | |
| wfg6 | 0.0560 | **0.0552** | 1.000 | 0.46 | |
| wfg7 | 0.0053 | **0.0053** | 1.000 | 0.54 | |
| wfg8 | 0.1394 | **0.1388** | 1.000 | 0.50 | |
| wfg9 | **0.0179** | 0.0218 | 1.000 | 0.38 | |

TABLE XIV
NORMALIZED HYPERVOLUME EQUIVALENCE SUMMARY (PAIRED BOOTSTRAP 90% CI; EQUIVALENCE MARGIN $\pm 1\%$ RELATIVE)

| Framework | Eq. problems | Non-inf. problems | Median $\Delta$HV (%) |
|---|---|---|---|
| pymoo | 16/21 | 19/21 | +0.00 |
| jMetalPy | 15/21 | 18/21 | +0.00 |

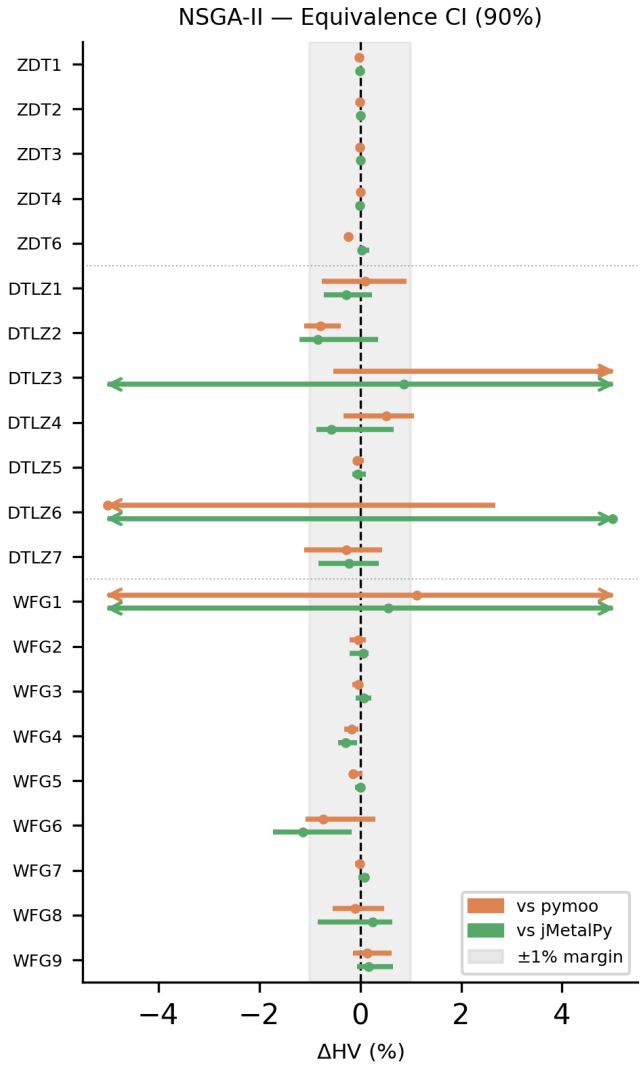


Fig. 9. NSGA-II. Per-problem paired bootstrap 90% confidence intervals for the relative normalized hypervolume difference $\Delta = (\mathrm{HV}_{VAMOS} - \mathrm{HV}_{\mathrm{fw}})/\mathrm{HV}_{\mathrm{fw}}$, reported in percent. The shaded band marks the $\pm 1\%$ equivalence margin; intervals falling entirely within this band indicate statistical equivalence

TABLE XV
ROBUSTNESS SUMMARY FOR NORMALIZED HYPERVOLUME ACROSS SEEDS (MEDIAN ACROSS PROBLEMS). "NEAR-BEST" IS DEFINED PER PROBLEM AS $\mathrm{HV} \geq 0.99 \cdot \max \mathrm{MEDIAN(HV)}$ AMONG FRAMEWORKS

| Framework | Median IQR(HV) | Median % seeds near-best |
|---|---|---|
| VAMOS | 0.001 | 100.0 |
| pymoo | 0.001 | 100.0 |
| jMetalPy | 0.001 | 100.0 |

TABLE XVI
MOEA/D. NORMALIZED HYPERVOLUME COMPARISON WITH WILCOXON SIGNED-RANK TEST RESULTS (HOLM-CORRECTED)

| Problem | VAMOS | pymoo | p-value | $\hat{A}_{12}$ | Sig. |
|---|---|---|---|---|---|
| zdt1 | **0.976** | 0.955 | ¡0.0001 | 0.99 | ✓ |
| zdt2 | **0.962** | 0.929 | ¡0.0001 | 0.96 | ✓ |
| zdt3 | **0.977** | 0.956 | ¡0.0001 | 0.99 | ✓ |
| zdt4 | **0.999** | 0.996 | 0.040 | 0.73 | ✓ |
| zdt6 | **0.991** | 0.945 | ¡0.0001 | 1.00 | ✓ |
| dtlz1 | **0.937** | 0.933 | 1.000 | 0.57 | |
| dtlz2 | **0.824** | 0.764 | ¡0.0001 | 1.00 | ✓ |
| dtlz3 | 0.000 | **0.000** | 0.061 | 0.64 | |
| dtlz4 | 0.803 | **0.805** | 1.000 | 0.45 | |
| dtlz5 | **0.879** | 0.878 | 0.318 | 0.62 | |
| dtlz6 | **0.924** | 0.918 | ¡0.0001 | 0.97 | ✓ |
| dtlz7 | **0.708** | 0.683 | 0.042 | 0.71 | ✓ |
| wfg1 | **0.204** | 0.199 | ¡0.0001 | 0.92 | ✓ |
| wfg2 | **0.892** | 0.846 | 0.115 | 0.60 | |
| wfg3 | **0.955** | 0.940 | ¡0.0001 | 0.90 | ✓ |
| wfg4 | **0.756** | 0.740 | 0.001 | 0.84 | ✓ |
| wfg5 | **0.809** | 0.793 | ¡0.0001 | 0.85 | ✓ |
| wfg6 | **0.703** | 0.692 | 0.318 | 0.75 | |
| wfg7 | **0.889** | 0.854 | ¡0.0001 | 0.98 | ✓ |
| wfg8 | **0.662** | 0.658 | 1.000 | 0.51 | |
| wfg9 | 0.712 | **0.849** | 0.125 | 0.38 | |

TABLE XVII
MOEA/D. IGD+ COMPARISON WITH WILCOXON SIGNED-RANK TEST RESULTS (HOLM-CORRECTED). LOWER IS BETTER

| Problem | VAMOS | pymoo | p-value | $\hat{A}_{12}$ | Sig. |
|---|---|---|---|---|---|
| zdt1 | **0.0091** | 0.0181 | ¡0.0001 | 0.01 | ✓ |
| zdt2 | **0.0069** | 0.0139 | ¡0.0001 | 0.03 | ✓ |
| zdt3 | **0.0060** | 0.0106 | ¡0.0001 | 0.01 | ✓ |
| zdt4 | **0.0012** | 0.0018 | 0.041 | 0.27 | ✓ |
| zdt6 | **0.0016** | 0.0106 | ¡0.0001 | 0.00 | ✓ |
| dtlz1 | **0.0197** | 0.0206 | 1.000 | 0.44 | |
| dtlz2 | **0.0421** | 0.0560 | ¡0.0001 | 0.00 | ✓ |
| dtlz3 | **3.0996** | 22.4201 | 0.145 | 0.24 | |
| dtlz4 | 0.0467 | **0.0464** | 1.000 | 0.54 | |
| dtlz5 | **0.0107** | 0.0108 | 0.439 | 0.40 | |
| dtlz6 | **0.0097** | 0.0100 | ¡0.0001 | 0.14 | ✓ |
| dtlz7 | **0.0765** | 0.0817 | 0.236 | 0.30 | |
| wfg1 | 1.1825 | **1.1807** | 1.000 | 0.58 | |
| wfg2 | **0.3212** | 0.3533 | 0.262 | 0.42 | |
| wfg3 | **0.0342** | 0.0470 | ¡0.0001 | 0.09 | ✓ |
| wfg4 | **0.1006** | 0.1103 | ¡0.0001 | 0.16 | ✓ |
| wfg5 | **0.0777** | 0.0864 | ¡0.0001 | 0.09 | ✓ |
| wfg6 | **0.1252** | 0.1322 | 0.440 | 0.26 | |
| wfg7 | **0.0410** | 0.0570 | ¡0.0001 | 0.03 | ✓ |
| wfg8 | **0.1980** | 0.2028 | 0.440 | 0.37 | |
| wfg9 | 0.1265 | **0.0721** | 0.262 | 0.60 | |

TABLE XVIII
NORMALIZED HYPERVOLUME EQUIVALENCE SUMMARY (PAIRED BOOTSTRAP 90% CI; EQUIVALENCE MARGIN $\pm 1\%$ RELATIVE)

| Framework | Eq. problems | Non-inf. problems | Median ΔHV (%) |
|---|---|---|---|
| pymoo | 5/21 | 18/21 | +2.15 |
| jMetalPy | 10/21 | 16/21 | +0.05 |

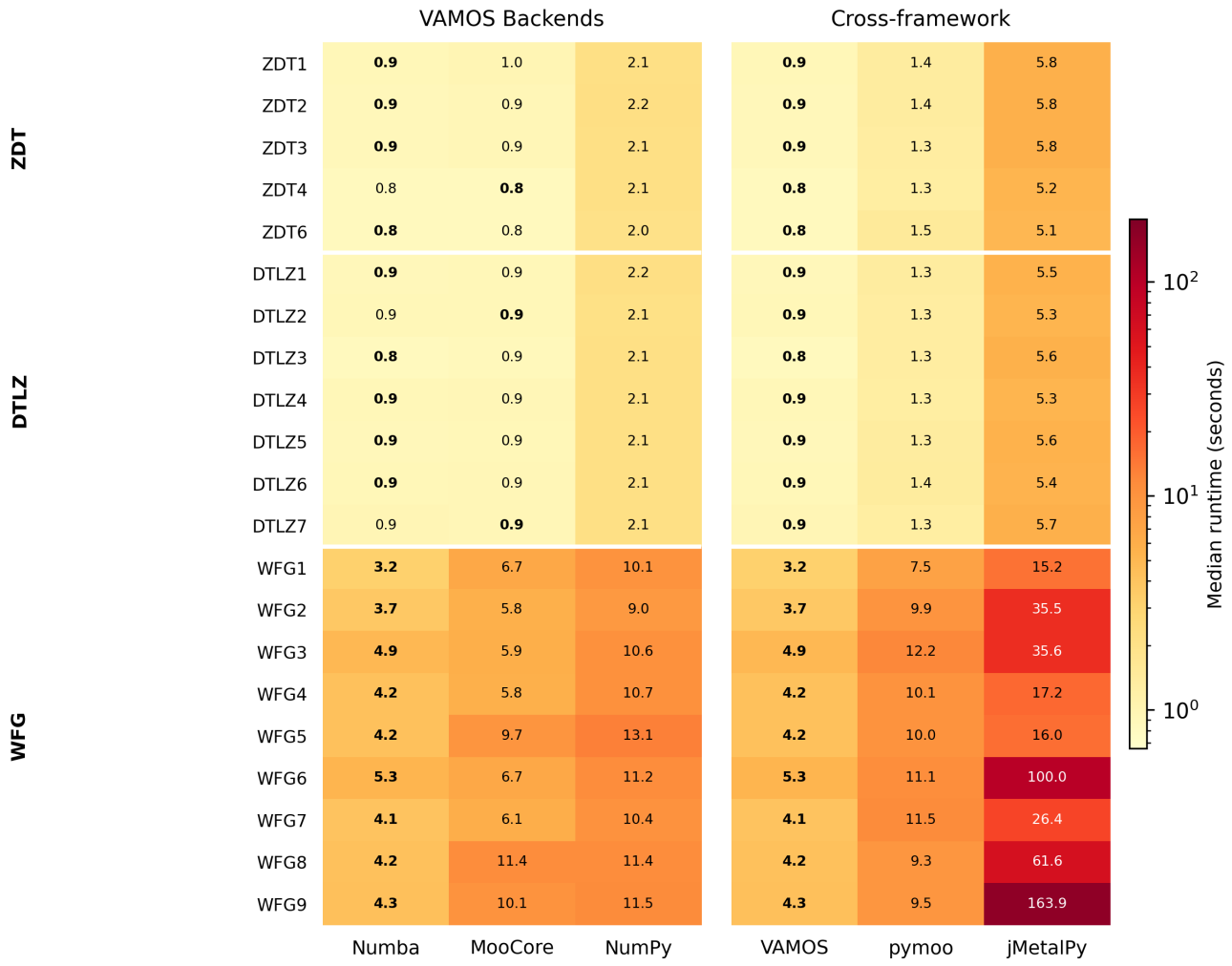


Fig. 10. Per-problem median runtime (seconds) for VAMOS backends (left) and cross-framework comparison (right). Color scale is logarithmic; bold values indicate the fastest in each row. Problems are grouped by family (ZDT, DTLZ, WFG).

TABLE XIX
ROBUSTNESS SUMMARY FOR NORMALIZED HYPERVOLUME ACROSS SEEDS (MEDIAN ACROSS PROBLEMS). "NEAR-BEST" IS DEFINED PER PROBLEM AS HV $\geq 0.99 \cdot \max$ MEDIAN(HV) AMONG FRAMEWORKS

| Framework | Median IQR(HV) | Median % seeds near-best |
|---|---|---|
| VAMOS | 0.013 | 73.3 |
| pymoo | 0.015 | 23.3 |
| jMetalPy | 0.008 | 83.3 |

## APPENDIX E
## ALGORITHM CONFIGURATION SPACES

Table XX provides a compact overview of the nine MOEAs implemented in VAMOS and their configurable parameter spaces across all supported encodings. Each row summarizes the algorithm's selection paradigm, supported encodings, number of available crossover and mutation operators, selection methods, algorithm-specific parameters, and total parameter count (including conditional parameters). The parameter counts reflect the tuning configuration spaces used by the built-in autotuner (Section III-A) and range from 8 (SMPSO) to 37 (NSGA-II), illustrating the breadth of component-level configurability that VAMOS exposes as first-class configuration options.

Table XXI summarizes the full parameter space considered for NSGA-II in VAMOS, illustrating the depth of component-level configurability described in Section III-A. The default configuration used throughout the experimental evaluation (Section IV) corresponds to the values shown in bold.

## APPENDIX F
## CODE EXAMPLES

Listing 4 shows how to autotune the full NSGA-II parameter space (Table XXI) using the Optuna backend. The `build_nsgaii_config_space` function returns the 28-parameter space; `ModelBasedTuner` drives the search with TPE sampling and optional multi-fidelity evaluation.

## REFERENCES

[1] K. Deb, *Multi-Objective Optimization Using Evolutionary Algorithms*. Chichester, UK: John Wiley & Sons, 2001.

TABLE XX

OVERVIEW OF THE NINE MOEAS IMPLEMENTED IN VAMOS AND THEIR CONFIGURABLE PARAMETER SPACES ACROSS ALL SUPPORTED ENCODINGS. ENCODINGS: R = REAL, P = PERMUTATION, B = BINARY, I = INTEGER, M = MIXED. THE "#XOVER" AND "#MUT" COLUMNS REPORT TOTALS ACROSS SUPPORTED ENCODINGS FOR EACH ALGORITHM. THE "#PARAMS" COLUMN REPORTS THE TOTAL NUMBER OF CONFIGURABLE PARAMETERS (BASE + CONDITIONAL) IN THE TUNING CONFIGURATION SPACE. ACRONYMS: SBX (SIMULATED BINARY CROSSOVER), PM (POLYNOMIAL MUTATION), DE (DIFFERENTIAL EVOLUTION), PBI (PENALTY-BASED BOUNDARY INTERSECTION).

| Algorithm | Paradigm | Encodings | #Xover | #Mut | Specific Parameters | #Params |
|---|---|---|---|---|---|---|
| NSGA-II | Dominance + crowding | R, P, B, I, M | 25 | 24 | Offspring ratio, external archive (type, size, bounded) | 37 |
| AGE-MOEA | Adaptive geometry | R, P, B, I, M | 21 | 22 | Archive policies (crowding, HV, $\varepsilon$-grid, hybrid) | 26 |
| RVEA | Reference vectors | R, P, B, I, M | 21 | 22 | Partitions, $\alpha$ decay, adaptation frequency | 28 |
| MOEA/D | Decomposition | R, P, B, I, M | 17 | 20 | Aggregation (Tchebycheff, WS, PBI), $T$, $\delta$, $\eta$ | 18 |
| IBEA | Indicator ($\varepsilon$/HV) | R, P, B, I, M | 21 | 22 | Indicator type, scaling factor $\kappa$ | 24 |
| SPEA2 | Strength Pareto | R, P, B, I, M | 21 | 22 | Archive size, $k$-nearest neighbors | 24 |
| NSGA-III | Reference directions | R, P, B, I, M | 21 | 22 | Reference directions (auto-computed) | 22 |
| SMS-EMOA | HV contribution | R, P, B, I, M | 21 | 22 | Reference point (adaptive) | 22 |
| SMPSO | Particle swarm | R, M | — | 3 | Inertia $\omega$, $c_1$, $c_2$, $v_{\max}$ fraction | 8 |

TABLE XXI

SUMMARY OF THE CONSIDERED PARAMETER SPACE FOR NSGA-II (REAL-VALUED ENCODING). PARAMETERS IN ITALICS ARE CONDITIONAL, ACTIVE ONLY WHEN THEIR CORRESPONDING OPERATOR IS SELECTED. VALUES IN **BOLD** INDICATE THE DEFAULT CONFIGURATION USED IN SECTION IV. THE TOTAL NUMBER OF CONFIGURABLE PARAMETERS IS 37. ACRONYMS: LHS (LATIN HYPERCUBE SAMPLING), SBX (SIMULATED BINARY CROSSOVER), BLX (BLEND CROSSOVER), PCX (PARENT CENTRIC CROSSOVER), UNDX (UNIMODAL NORMAL DISTRIBUTION CROSSOVER), SPX (SIMPLEX CROSSOVER), PM (POLYNOMIAL MUTATION), SUS (STOCHASTIC UNIVERSAL SAMPLING).

| Component | Value / Strategy | Conditional Parameters |
|---|---|---|
| General | Pop. Size $\in [20, 200]_{\mathbb{Z}}$ (log-scale); default **100** | – |
| | Offspring Ratio $\in \{0.25, 0.5, 0.75, \mathbf{1.0}\}$ | – |
| | Selection $\in \{$**Tournament**, Boltzmann, Ranking, SUS$\}$ | Pressure $\in [\mathbf{2}, 10]_{\mathbb{Z}}$ |
| | Use External Archive $\in \{$True, **False**$\}$ | – |
| | *Archive Unbounded* $\in \{$True, False$\}$ | *Archive* = True |
| | *Archive Type* $\in \{$size_cap, hvc_prune$\}$ | *Unbounded* = False |
| | *Archive Size Factor* $\in \{1, 2, 5, 10\}$ | *Unbounded* = False |
| Initialization | $\{$**Random**, LHS, Scatter$\}$ | – |
| | *Scatter Base Size Factor* $\in \{0.1, 0.2, 0.3, 0.5, 0.75, 1.0\}$ | *Init.* = Scatter |
| Crossover | *Global:* Probability $\in [0.6, 1.0]_{\mathbb{R}}$, Repair $\in \{$none, **clip**, reflect, random, round$\}$ | |
| | **SBX** | *Dist. Index* $\in [5.0, 40.0]_{\mathbb{R}}$ |
| | BLX-$\alpha$ | $\alpha \in [0.0, 1.0]_{\mathbb{R}}$, *Repair* $\in \{$clip, random, reflect, round$\}$ |
| | BLX-$\alpha\beta$ | $\alpha \in [0.0, 1.0]_{\mathbb{R}}$, $\beta \in [0.0, 1.0]_{\mathbb{R}}$ |
| | Arithmetic | – |
| | WholeArithmetic | $\alpha \in [0.0, 1.0]_{\mathbb{R}}$ |
| | Laplace | $a \in [-1.0, 1.0]_{\mathbb{R}}$, $b \in [0.01, 2.0]_{\mathbb{R}}$ |
| | Fuzzy | $d \in [0.0, 2.0]_{\mathbb{R}}$ |
| | PCX | $\sigma_\eta \in [0.01, 0.5]_{\mathbb{R}}$, $\sigma_\zeta \in [0.01, 0.5]_{\mathbb{R}}$ |
| | UNDX | $\zeta \in [0.1, 1.0]_{\mathbb{R}}$, $\eta \in [0.1, 1.0]_{\mathbb{R}}$ |
| | Simplex | $\epsilon \in [0.1, 1.0]_{\mathbb{R}}$ |
| Mutation | *Global:* Prob. Factor $\in [0.25, 3.0]_{\mathbb{R}}$ ($p_m$ = factor/$n$), Dist. Index $\in [5.0, 40.0]_{\mathbb{R}}$ | |
| | **PM** | (uses global dist. index) |
| | LinkedPolynomial | (uses global dist. index) |
| | NonUniform | *Perturbation* $\in [0.05, 0.5]_{\mathbb{R}}$ |
| | Gaussian | $\sigma \in [0.001, 0.5]_{\mathbb{R}}$ |
| | Cauchy | $\gamma \in [0.001, 0.5]_{\mathbb{R}}$ |
| | Uniform | *Perturbation* $\in [0.01, 0.5]_{\mathbb{R}}$ |
| | UniformReset | – |
| | LévyFlight | $\beta \in [0.5, 2.0]_{\mathbb{R}}$, *Scale* $\in [0.001, 0.1]_{\mathbb{R}}$ |
| | PowerLaw | *Index* $\in [0.5, 5.0]_{\mathbb{R}}$ |

Listing 1. Minimal NSGA-II run on ZDT1 in VAMOS.

```
from vamos import optimize
result = optimize("zdt1", algorithm="nsgaii",
                  max_evaluations=50000, seed=42)
```

Listing 2. Builder API for fine-grained configuration.

```
from vamos import optimize
from vamos.algorithms import NSGAIIConfig
cfg = (NSGAIIConfig.builder()
       .pop_size(100)
       .crossover("sbx", prob=1.0, eta=20)
       .mutation("pm", prob="1/n", eta=20)
       .build())
result = optimize("zdt1", algorithm="nsgaii",
                  algorithm_config=cfg,
                  max_evaluations=50000, seed=42)
```

Listing 3. Defining and optimizing a custom problem.

```
from vamos import make_problem, optimize
problem = make_problem(
    lambda x: [x[0]**2 + x[1]**2,
               (x[0]-1)**2 + (x[1]-1)**2],
    n_var=2, n_obj=2, bounds=[(0, 2), (0, 2)])
result = optimize(problem, algorithm="nsgaii",
                  max_evaluations=50000, seed=42)
```

Listing 4. Autotuning NSGA-II with Optuna on ZDT1.

```
import numpy as np
from vamos import optimize
from vamos.engine.tuning import (
    ModelBasedTuner, TuningTask, Instance, EvalContext,
    build_nsgaii_config_space, config_from_assignment,
)

# Build the 28-parameter NSGA-II config space
space = build_nsgaii_config_space()

# Evaluation function: returns hypervolume
def eval_fn(config, ctx: EvalContext):
    cfg = config_from_assignment("nsgaii", config)
    res = optimize("zdt1", algorithm="nsgaii",
                   algorithm_config=cfg,
                   max_evaluations=ctx.budget, seed=ctx.seed)
    return float(res.hypervolume(ref_point=[1.1, 1.1]))

# Define tuning task
task = TuningTask(
    name="tune_nsgaii_zdt1",
    param_space=space.to_param_space(),
    instances=[Instance(name="zdt1", n_var=30)],
    seeds=[1, 2, 3], aggregator=np.mean,
    budget_per_run=10000, maximize=True,
)

# Run Optuna-based tuning (TPE + multi-fidelity)
tuner = ModelBasedTuner(
    task=task, max_trials=50, backend="optuna",
    seed=42, n_jobs=4,
    budget_levels=[2000, 5000, 10000],
)
best_config, history = tuner.run(eval_fn)
```

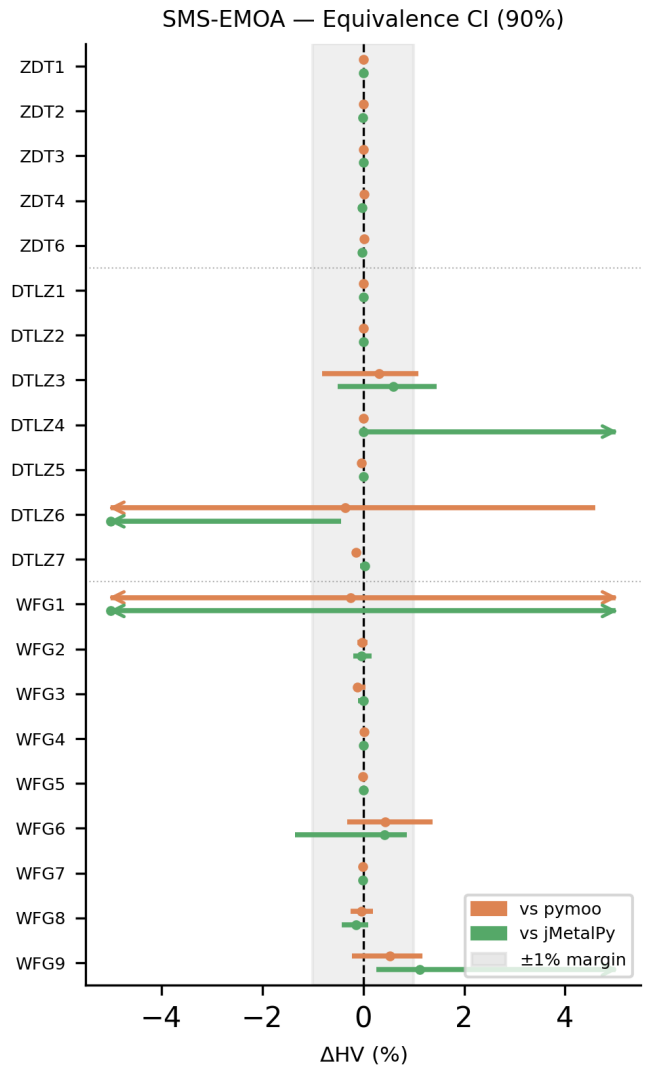


Fig. 11. SMS-EMOA. Per-problem paired bootstrap 90% confidence intervals for the relative normalized hypervolume difference $\Delta = (\mathrm{HV}_{VAMOS} - \mathrm{HV}_{\mathrm{fw}})/\mathrm{HV}_{\mathrm{fw}}$, reported in percent. The shaded band marks the $\pm 1\%$ equivalence margin; intervals falling entirely within this band indicate statistical equivalence

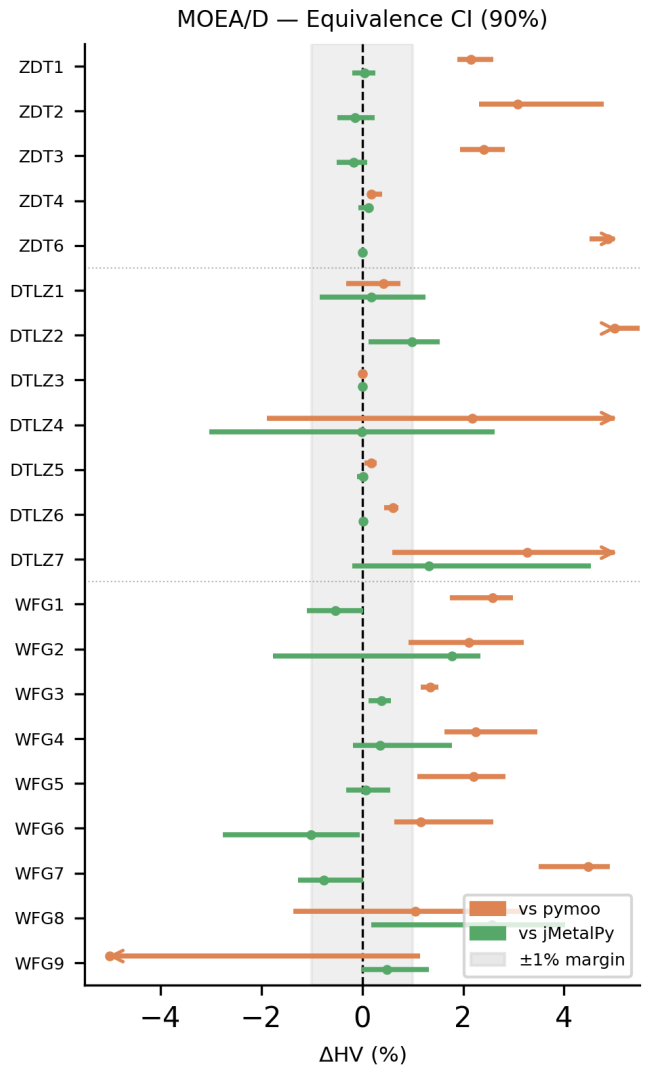


Fig. 12. MOEA/D. Per-problem paired bootstrap 90% confidence intervals for the relative normalized hypervolume difference $\Delta = (\mathrm{HV}_{VAMOS} - \mathrm{HV}_{\mathrm{fw}})/\mathrm{HV}_{\mathrm{fw}}$, reported in percent. The shaded band marks the $\pm 1\%$ equivalence margin; intervals falling entirely within this band indicate statistical equivalence


[2] C. A. C. Coello, G. B. Lamont, and D. A. V. Veldhuizen, *Evolutionary Algorithms for Solving Multi-Objective Problems*, 2nd ed. New York, NY, USA: Springer, 2007.

[3] S. Bleuler, M. Laumanns, L. Thiele, and E. Zitzler, "PISA—a platform and programming language independent interface for search algorithms," in *Proc. Evolutionary Multi-Criterion Optimization (EMO)*, ser. Lecture Notes in Computer Science, vol. 2632. Springer, 2003, pp. 494–508.

[4] J. J. Durillo and A. J. Nebro, "jMetal: A Java framework for multi-objective optimization," *Advances in Engineering Software*, vol. 42, no. 10, pp. 760–771, 2011.

[5] Y. Tian, R. Cheng, X. Zhang, and Y. Jin, "PlatEMO: A MATLAB platform for evolutionary multi-objective optimization," *IEEE Computational Intelligence Magazine*, vol. 12, no. 4, pp. 73–87, 2017.

[6] J. Blank and K. Deb, "pymoo: Multi-objective optimization in Python," *IEEE Access*, vol. 8, pp. 89 497–89 509, 2020.

[7] A. Benítez-Hidalgo, A. J. Nebro, J. García-Nieto, I. Oregi, and J. Del Ser, "jMetalPy: A Python framework for multi-objective optimization with metaheuristics," *Swarm and Evolutionary Computation*, vol. 51, p. 100598, 2019.

[8] D. Hadka, "Platypus: A free and open source Python library for multiobjective optimization," https://github.com/Project-Platypus/Platypus, accessed: 2026.

[9] F.-A. Fortin, F.-M. De Rainville, M.-A. Gardner, M. Parizeau, and C. Gagné, "DEAP: Evolutionary algorithms made easy," *Journal of Machine Learning Research*, vol. 13, pp. 2171–2175, 2012.

[10] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: NSGA-II," *IEEE Transactions on Evolutionary Computation*, vol. 6, no. 2, pp. 182–197, 2002.

[11] H. Ishibuchi, L. M. Pang, and K. Shang, "A new framework of evolutionary multi-objective algorithms with an unbounded external archive," in *ECAI 2020 - 24th European Conference on Artificial Intelligence, 29 August-8 September 2020, Santiago de Compostela, Spain, August 29 - September 8, 2020 - Including 10th Conference on Prestigious Applications of Artificial Intelligence (PAIS 2020)*, ser. Frontiers in Artificial Intelligence and Applications, G. D. Giacomo, A. Catalá, B. Dilkina, M. Milano, S. Barro, A. Bugarín, and J. Lang, Eds., vol. 325. IOS Press, 2020, pp. 283–290.

[12] F. Biscani and D. Izzo, "A parallel global multiobjective framework for optimization: pagmo," *Journal of Open Source Software*, vol. 5, no. 53, p. 2338, 2020. [Online]. Available: https://doi.org/10.21105/joss.02338

[13] B. Huang, R. Cheng, Z. Li, Y. Jin, and K. C. Tan, "EvoX: A distributed GPU-accelerated framework for scalable evolutionary computation," *IEEE Transactions on Evolutionary Computation*, vol. 29, no. 5, pp. 1649–1662, 2025.

[14] Y. Tian, T. Zhang, J. Xiao, X. Zhang, and Y. Jin, "A coevolutionary framework for constrained multiobjective optimization problems," *IEEE Transactions on Evolutionary Computation*, vol. 25, no. 1, pp. 102–116, Feb. 2021.

[15] H. Zhao, X.-H. Ning, J.-Y. Li, and J. Liu, "Evolutionary multitask framework with bi-knowledge transfer for multimodal optimization problems," *IEEE Transactions on Evolutionary Computation*, vol. 30, no. 1, pp. 393–407, Feb. 2026.

[16] H. Ye, H. Xu, and C. A. Coello Coello, "Light-EvoOPT: A lightweight evolutionary optimization framework for ultralarge-scale mixed integer linear programs," *IEEE Transactions on Evolutionary Computation*, vol. 30, no. 1, pp. 333–347, Feb. 2026.

[17] Z. Ma, Y.-J. Gong, H. Guo, J. Chen, Y. Ma, Z. Cao, and J. Zhang, "LLaMoCo: Instruction tuning of large language models for optimization code generation," *IEEE Transactions on Evolutionary Computation*, 2026, early access.

[18] M. L.-I. nez, J. Dubois-Lacoste, L. Pérez Cáceres, T. Stützle, and M. Birattari, "The irace package: Iterated racing for automatic algorithm configuration," *Operations Research Perspectives*, vol. 3, pp. 43–58, 2016.

[19] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proc. ACM SIGKDD Int. Conf. Knowl. Discov. Data Min. (KDD)*, 2019, pp. 2623–2631.

[20] M. Lindauer, K. Eggensperger, M. Feurer, A. Biedenkapp, D. Deng, C. Benjamins, T. Ruhkopf, R. Sass, and F. Hutter, "SMAC3: A versatile Bayesian optimization package for hyperparameter optimization," *Journal of Machine Learning Research*, vol. 23, no. 54, pp. 1–9, 2022.

[21] S. Falkner, A. Klein, and F. Hutter, "BOHB: Robust and efficient hyperparameter optimization at scale," in *Proc. Int. Conf. Mach. Learn. (ICML)*, ser. PMLR, vol. 80, 2018, pp. 1437–1446.

[22] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Krevelen, M. Brett, A. Haldane, J. Fernández del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, and T. E. Oliphant, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020.

[23] S. K. Lam, A. Pitrou, and S. Seibert, "Numba: A LLVM-based Python JIT compiler," in *Proc. 2nd Workshop on the LLVM Compiler Infrastructure in HPC (LLVM-HPC)*, 2015.

[24] M. L.-I. nez *et al.*, "moocore: Multi-objective optimization core library," https://github.com/multi-objective/moocore, accessed: 2026.

[25] K. Deb and H. Jain, "An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, part I: Solving problems with box constraints," *IEEE Transactions on Evolutionary Computation*, vol. 18, no. 4, pp. 577–601, 2014.

[26] Q. Zhang and H. Li, "MOEA/D: A multiobjective evolutionary algorithm based on decomposition," *IEEE Transactions on Evolutionary Computation*, vol. 11, no. 6, pp. 712–731, 2007.

[27] N. Beume, B. Naujoks, and M. Emmerich, "SMS-EMOA: Multiobjective selection based on dominated hypervolume," *European Journal of Operational Research*, vol. 181, no. 3, pp. 1653–1669, 2007.

[28] E. Zitzler, M. Laumanns, and L. Thiele, "SPEA2: Improving the strength Pareto evolutionary algorithm," ETH Zurich, Zurich, Switzerland, Tech. Rep. TIK-Report 103, 2001.

[29] E. Zitzler and S. Künzli, "Indicator-based selection in multiobjective search," in *Proc. Int. Conf. Parallel Problem Solving from Nature (PPSN)*. Springer, 2004, pp. 832–842.

[30] A. J. Nebro, J. J. Durillo, and C. A. Coello Coello, "SMPSO: A new PSO-based metaheuristic for multi-objective optimization," in *Proc. IEEE Symp. Comput. Intell. Multi-Criteria Decision-Making*, 2009, pp. 66–73.

[31] A. Panichella, "An adaptive evolutionary algorithm based on non-Euclidean geometry for many-objective optimization," in *Proc. Genetic and Evolutionary Computation Conf. (GECCO)*. ACM, 2019, pp. 595–603.

[32] R. Cheng, Y. Jin, M. Olhofer, and B. Sendhoff, "A reference vector guided evolutionary algorithm for many-objective optimization," *IEEE Transactions on Evolutionary Computation*, vol. 20, no. 5, pp. 773–791, 2016.

[33] E. Zitzler, K. Deb, and L. Thiele, "Comparison of multiobjective evolutionary algorithms: Empirical results," *Evolutionary Computation*, vol. 8, no. 2, pp. 173–195, 2000.

[34] K. Deb, L. Thiele, M. Laumanns, and E. Zitzler, "Scalable multi-objective optimization test problems," in *Proc. Congr. Evol. Comput. (CEC)*, 2002, pp. 825–830.

[35] S. Huband, P. Hingston, L. Barone, and L. While, "A review of multiobjective test problems and a scalable test problem toolkit," *IEEE Transactions on Evolutionary Computation*, vol. 10, no. 5, pp. 477–506, 2006.

[36] E. Zitzler, L. Thiele, M. Laumanns, C. M. Fonseca, and V. Grunert da Fonseca, "Performance assessment of multiobjective optimizers: An analysis and review," *IEEE Transactions on Evolutionary Computation*, vol. 7, no. 2, pp. 117–132, 2003.

[37] H. Ishibuchi, H. Masuda, Y. Tanigaki, and Y. Nojima, "Modified distance calculation in generational distance and inverted generational distance," in *Evolutionary Multi-Criterion Optimization (EMO)*, ser. Lecture Notes in Computer Science, vol. 9019. Springer, 2015, pp. 110–125.

[38] A. Vargha and H. D. Delaney, "A critique and improvement of the CL common language effect size statistics of McGraw and Wong," *Journal of Educational and Behavioral Statistics*, vol. 25, no. 2, pp. 101–132, 2000.